

Мультиагентная система для ресёрча ниши на Wildberries: проектирование, реализация и оценка

Отчёт по проектной работе

Идрисов Данияр Фаритович, группа 11-561

2026

Автор: Идрисов Данияр Фаритович, группа 11-561

Репозиторий проекта: <https://github.com/DaniyarIdrisov/niche-research-agent>

Документация проекта: <https://github.com/DaniyarIdrisov/niche-research-agent/tree/main/docs>

Аннотация

В работе спроектирована и реализована **мультиагентная система** для автоматизации первичного ресёрча ниши на маркетплейсе Wildberries в категории малой бытовой техники (МБТ). Система принимает запрос пользователя на естественном языке (например, *«оцени нишу — электрические зубные щётки до 3000 рублей»*) и возвращает аналитический отчёт по нише, формализованный Product Requirements Document (PRD) на новый товар, анализ уникальных торговых предложений конкурентов и итоговый вердикт go / no-go.

Архитектура построена по паттерну **Supervisor + специализированные воркеры** на фреймворке **LangGraph**, включает шесть агентов: Scout, Niche Analyst, Specs Miner, USP Analyst, PRD Writer, Reporter. Параллельный fan-out трёх аналитиков реализован через нативный механизм LangGraph (возврат списка целей из conditional router). Каждая критичная нода защищена retry-петлёй с ограничением попыток.

Реализована **трёхуровневая система памяти**: working (LangGraph state), episodic (SQLite журнал запусков), semantic (ChromaDB + BM25 hybrid retrieval с опциональным перанкером bge-reranker-v2-m3). Языковая модель — **Qwen 2.5 3B Instruct** в квантизации Q4_K_M через Ollama; эмбединги — **nomic-embed-text**. Жёсткое аппаратное ограничение в 4 ГБ VRAM (NVIDIA GTX 1050 Ti) полностью соблюдено.

Внедрён полный стек observability: **OpenTelemetry** (трейсы → Jaeger), **Prometheus + Grafana** (метрики, 8 типов counter/histogram), **Loguru** (структурированные JSON-логи), **Langfuse** (LLM-специфичный трейсинг). Реализован полнофункциональный фреймворк оценки: компонентные и системные evals, LLM-as-judge, Jupyter-ноутбук с визуализацией результатов.

Объём проекта: **87 файлов, более 12 000 строк** кода и документации. Все шесть фаз разработки задокументированы, для каждого архитектурного решения зафиксированы альтернативы и trade-offs.

Содержание

Содержание

Аннотация

Содержание

Введение

Проблема

Решение

Ограничения

Анализ предметной области

Специфика Wildberries

Специфика МБТ

Целевая аудитория

Цели и задачи

Цель работы

Функциональные требования (ФТ)

Нефункциональные требования (НФТ)

Out of scope

Ход работы

Фаза 1: Каркас проекта и инфраструктура

Фаза 2: Mock-датасет и эталонные запросы

Фаза 3: MVP — первый рабочий pipeline

Фаза 4: Оставшиеся агенты, память, скиллы

Фаза 5: RAG, knowledge base, evals

Фаза 6: Observability и финальная документация

Итоговая статистика проекта

Архитектура системы

Высокоуровневая схема

Поток данных одного запроса

Граф LangGraph как кодовая структура

Компоненты по слоям

Технологический стек: обоснование выборов

Языковая модель: Qwen 2.5 3B Instruct Q4_K_M

Эмбединг-модель: nomic-embed-text

Inference: Ollama (нативно на хосте)

Фреймворк агентов: LangGraph

Векторное хранилище: ChromaDB

Episodic memory: SQLite

Hybrid retrieval: ChromaDB (dense) + rank_bm25 (sparse)

Реранкер: bge-reranker-v2-m3 (опционально)

LLM-клиент: голый httpх

Observability стек

CLI: Typer + Rich

Менеджер пакетов: uv

Контейнеризация: Docker Compose

Архитектурный стиль и нотация

Парадигма: Multi-agent + Supervisor

Нотация диаграмм

Структура промптов

Принципы кодирования

Память и RAG: детальный обзор

Три уровня памяти

RAG: hybrid retrieval

Chunking стратегия

Тюнинг параметров retrieval

Скиллы (формат Anthropic Agent Skills)

Observability: 4 столпа

Поток данных

Метрики Prometheus

Grafana-дашборд

Сравнение с альтернативами

Изоляция

Что реализовано

Спектр вариантов и trade-off

Evaluations: методология и реализация

Уровни оценки

Пороги приёмки

Размеченные эталоны

LLM-as-judge: признание bias-риска

Демонстрация работы

Проверка готовности

Запуск ресёрча ниши

Observability в действии

Jaeger — трейсы

Grafana — метрики

Langfuse — LLM-tracing (текущий статус и границы реализации)

Prometheus — сырые метрики

Прогон evaluations

Component-level

System-level

Юрутер-ноутбук с визуализацией

Проблемы и решения

- Проблема 1: Нестабильный JSON-output на 3B-модели
- Проблема 2: LLM не справляется с арифметикой
- Проблема 3: Параллельный fan-out в LangGraph
- Проблема 4: Эмбеддер и LLM не помещаются вместе в 4 ГБ VRAM
- Проблема 5: cp1251 / UTF-8 на Windows
- Проблема 6: search.wb.ru — серая зона
- Проблема 7: Self-bias в LLM-as-judge
- Проблема 8: Graceful degradation observability

Результаты

- Количественные результаты
- Качественные результаты

Выводы и дальнейшая работа

- Что доказано
- Слабые места (явно)
- Дальнейшая работа
- Что хочется отметить особо

Источники и материалы

- Документация и спецификации
- Регламенты и нормативная база
- Технические статьи и репег'ы
- Wildberries-специфика

Приложение А. Структура файлов проекта

Приложение В. Снимок ключевых артефактов

Приложение В. Команды для воспроизведения

Что нужно для финализации отчёта в PDF

- Шаг 1. Подготовить скриншоты по списку
- Шаг 2. Положить скриншоты в папку
- Шаг 3. Заменить placeholder'ы
- Шаг 4. Сгенерировать PDF
- Список скриншотов: пронумерованный чек-лист
- Готовые URL для скринов Mermaid-диаграмм

Введение

Проблема

Категорийный менеджер маркетплейса перед запуском нового товара (или брендом перед выходом в нишу) тратит значительное время на ручной анализ:

- сбор топа карточек по поисковому запросу,
- подсчёт ёмкости ниши и концентрации топ-продавцов,
- извлечение и сравнение характеристик конкурентов,
- разбор маркетинговых формулировок (УТП) топов,
- проверку требований к сертификации (для МБТ: декларации ТР ТС, маркировка ЕАС),
- формирование задания на новый товар.

Эта работа повторяется для каждой ниши и плохо стандартизована — разные специалисты приходят к разным выводам на одних и тех же данных.

Решение

Автоматизировать пайплайн полностью локально, без обращений к облачным LLM-провайдерам, на потребительском железе. Применить парадигму **мультиагентных LLM-систем**: разбить задачу на ряд узких подзадач, для каждой — отдельный агент с собственным промптом, схемой структурированного вывода и фолбэками.

Ограничения

Жёсткое требование университетского проекта — **запуск на слабом железе**:

Параметр	Целевое значение
GPU VRAM	4 ГБ (NVIDIA GTX 1050 Ti)
RAM	16 ГБ
CPU	Intel i5 4-го поколения, 4 ядра
ОС	Windows 11 + WSL2 / Linux
Лицензия зависимостей	OSS / open-weights
Облачные сервисы	запрещены

Из этих ограничений вытекает большинство архитектурных решений (см. § 4).

Анализ предметной области

Специфика Wildberries

Wildberries — крупнейший по обороту маркетплейс в России. По состоянию на 2025 год площадка имеет:

- свыше 700 тыс. активных продавцов,
- 1.5+ млн SKU только в категории «Бытовая техника»,
- закрытый официальный API (доступ только продавцу к своим карточкам),
- публичный поисковый бэкенд `search.wb.ru` (без авторизации, без публичной документации, формат может меняться).

Это сразу формирует два требования к системе:

1. **Не полагаться только на live-режим парсинга.** Нужен mock-режим с реалистичным датасетом — иначе систему нельзя ни запустить на демонстрации, ни прогнать evals.
2. **Парсер должен быть устойчив к изменениям формата.** Реализован через маппинг исключительно тех полей, что нужны для расчётов (`salePriceU` , `feedbacks` , `rating`), а не всей структуры ответа.

Специфика МБТ

Категория малой бытовой техники имеет существенные особенности по сравнению, например, с одеждой:

Признак	МБТ	Одежда
Решающий фактор покупки	технические характеристики (мощность, режимы, насадки)	размер, материал, бренд
Стандартизация описаний	низкая (синонимы: «мощность» vs «потребляемая мощность»)	средняя
Обязательная сертификация	ДА (ТР ТС 004, ТР ТС 020)	нет
Длительность жизни SKU	1-3 года	сезон
Возвраты	до 8% (производственный брак, несоответствие описанию)	до 30% (размер)

Из этого следует:

- Извлечение характеристик — **отдельный шаг с нормализацией синонимов** (Specs Miner + Specs Normalizer).
- Обязательная секция compliance в PRD — нельзя игнорировать.
- УТП в категории чаще всего опираются на **числа и долговечность** (см. далее USP Analyst, тип `technological`).

Целевая аудитория

Первичный пользователь — **категорийный менеджер** маркетплейса (или продакт-менеджер бренда), принимающий решение «выходить или не выходить в нишу». Он:

- знаком с терминологией WB (СПП, оборот, ABC-XYZ, БРВ-1, FBO/FBS),
- доверяет агенту первичную аналитику, но финальное решение оставляет за собой,
- не хочет отправлять стратегические данные в облако,
- располагает рабочим ноутбуком, не серверной инфраструктурой.

Цели и задачи

Цель работы

Создать защитимый университетский проект, демонстрирующий компетенции в:

- проектировании мультиагентных LLM-систем,
- работе с фреймворком LangGraph,
- реализации RAG-памяти с гибридным retrieval,
- инструментировании систем наблюдаемостью,
- оценке агентных систем (component + system + LLM-as-judge),
- осознанном выборе технологического стека с учётом ограничений.

Функциональные требования (ФТ)

1. **ФТ-01.** Принимать запрос на естественном языке (категория + опциональные фильтры).
2. **ФТ-02.** Возвращать аналитический отчёт по нише: оборот топ-30, доля топ-5, медианная цена, спред, динамика отзывов.
3. **ФТ-03.** Возвращать PRD по фиксированному шаблону.
4. **ФТ-04.** Возвращать анализ УТП топ-5 с матрицей пустот.
5. **ФТ-05.** Выдавать вердикт по явному набору критериев.
6. **ФТ-06.** Поддерживать два режима данных: mock и live.
7. **ФТ-07.** Сохранять историю запусков с возможностью возврата по run-id.
8. **ФТ-08.** Логировать промпты, ответы и трейсы каждого агента.

Нефункциональные требования (НФТ)

Код	Требование	Целевое значение
НФТ-01	End-to-end latency p95	≤ 5 минут на 1050 Ti, mock
НФТ-02	LLM throughput	≥ 15 токенов/сек
НФТ-03	Cold start	≤ 30 сек (без загрузки моделей)
НФТ-04	Операционка	Windows + WSL2 / Linux
НФТ-05	Размер образов Docker	≤ 15 ГБ суммарно
НФТ-06	Все секреты	через .env , не в репо
НФТ-07	Graceful degradation	при падении одного компонента pipeline продолжает работать

Out of scope

Сознательно исключено:

- Публикация / редактирование карточек на WB.
- Финансовая модель целиком (только базовый юнит-калькулятор).

- Сезонность (нет исторических данных).
- Другие маркетплейсы (Ozon, Я.Маркет).
- Real-time мониторинг.

Ход работы

Разработка была разбита на **шесть фаз** с явными точками проверки между ними. Каждая фаза заканчивалась `python -m compileall` smoke-тестом и фиксацией списка готовых артефактов.

Фаза 1: Каркас проекта и инфраструктура

Сделано:

- Создана полная структура каталогов (`src/` , `prompts/` , `skills/` , `knowledge_base/` , `data/` , `evals/` , `docs/` , `observability/` , `tests/`).
- Написан `README.md` с пошаговой инструкцией запуска, разделом «Тонкости работы на 1050 Ti» и troubleshooting'ом.
- Сконфигурирован `pyproject.toml` (`uv + hatchling`, Python 3.11, разбиение на `core / dev / evals`).
- Поднят `docker-compose.yml` со всеми сервисами (ChromaDB, Jaeger, OTEL collector, Prometheus, Grafana, Langfuse + Postgres). **Ollama сознательно вне compose** — нативно на хосте, чтобы избежать потери производительности GPU на CUDA-прокидывании в контейнер.
- Описан изолированный контейнер `wb-parser` для live-режима: `read_only: true` , `tmpfs: /tmp` , `security_opt: no-new-privileges:true` , `cap_drop: ALL` .
- Подготовлены конфиги observability (`prometheus.yml` , `otel-collector-config.yaml` , Grafana provisioning).
- Составлено формальное ТЗ (`docs/TZ.md`) со всеми ФТ/НФТ.

Артефакты: 13 файлов конфигурации и документации.

Фаза 2: Mock-датасет и эталонные запросы

Сделано:

- Реализован детерминированный генератор `data/generate_mock_data.py` (только `stdlib`, `seed=42` для воспроизводимости).
- Сгенерированы **115 карточек** на 3 категории МБТ (40 электрических зубных щёток + 40 фенов + 35 эпиляторов).
- Заложены реалистичные распределения:
- **лог-нормальные цены** с разными `mean / sigma` по категориям;
- **бета-распределение рейтингов** (перекошено к 4.5+);
- **степенное распределение отзывов** (Парето, $k=1.6$) — реальный длинный хвост;
- **прокси-оценка продаж** через множитель к отзывам.
- Сознательно введены **синонимы ключей характеристик** («автономность» / «время работы» / «продолжительность работы») — материал для последующей нормализации.
- ~8% карточек оставлены без некоторых полей — реализм данных WB.
- В описания и заголовки встроены **четыре типа УТП**: технологические, value, эмоциональные, социальные — для агента USP Analyst.
- Составлен `data/eval_queries.json` — **15 эталонных запросов** с блоком `expected` : 8 happy-path, 2 edge case (пустая ниша, узкий фильтр), 2 ожидаемых вердикта, 1 compliance, 1 короткий нечёткий, 1 сравнение сегментов (OOS).

Проблема: при первом запуске генератора Python упал на `print()` финальной строки с символом `Р` из-за дефолтной cp1251-кодировки stdout на Windows. **Решение:** убраны не-ASCII символы из `print`, добавлен `$env:PYTHONUTF8="1"` в запуск.

Артефакты: генератор (~450 строк), JSON-датасет (115 карточек), JSON eval-набор (15 запросов).

Фаза 3: MVP — первый рабочий pipeline

Сделано:

- Реализован `src/config.py` — pydantic-settings, lazy singleton, чтение `.env`.
- Написан `src/llm/ollama_client.py` — собственная тонкая обёртка над Ollama HTTP API на голом httpx (без langchain-ollama). Включает:
 - `chat()` — низкоуровневый вызов с tenacity-retry на 5xx и network errors;
 - `chat_text()` / `chat_structured()` — высокоуровневые;
- **Repair-loop** в `chat_structured()`: при невалидном JSON показывает модели её собственный ответ и ошибку валидации, делает до 2 доп. попыток;
- `embed()` — батчинг для эмбединг-модели.
- Реализованы Pydantic-схемы: `Product`, `QueryFilters` (с авто-нормализацией `min > max`), `AgentState` как TypedDict.
- Написан `src/tools/wb_parser.py` с двумя режимами:
 - **mock** — читает локальный JSON, применяет фильтры, ранжирует;
 - **live** — обращается к `search.wb.ru/exactmatch/ru/common/v9/search` с token-bucket rate-limiter (по умолчанию 2 req/sec), маппит сырой ответ на `Product`.
- Реализован `src/tools/calculator.py` — детерминированные функции: `revenue_top_n`, `seller_concentration`, `price_stats` (с p25/p75), `review_dynamics`, `summarize`.
- Создан первый агент **Scout** с системным промптом в `prompts/scout.md` (6 few-shot примеров, включая edge cases).
- **Ключевое архитектурное решение:** Scout *не* генерирует список товаров. LLM парсит только NL-запрос в `QueryFilters`; список карточек возвращается детерминированно из `wb_parser.search_products()`. Это убирает риск галлюцинаций SKU и цен на 3B-модели.
- Реализован минимальный LangGraph supervisor: `START` → `scout` → `validate_scout` → `END` с conditional edge на retry.
- Написан CLI на Typer + Rich (`src/main.py`): команды `research`, `check`, `history list`, `history show`.

Артефакт: 13 новых модулей. `python -m compileall src/` → exit=0.

Фаза 4: Оставшиеся агенты, память, скиллы

Сделано:

- Реализованы **4 уровня памяти**:
 - `src/memory/working.py` — хелперы поверх state (`add_error`, `increment_retry`);
 - `src/memory/episodic.py` — sqlite3 с таблицей `runs(run_id, query, started_at, finished_at, verdict, n_products, state_json, errors_json)`;
 - `src/memory/semantic.py` — заглушка с правильным API (полная реализация в Фазе 5).

- Реализованы Pydantic-схемы структурированного вывода: `NicheMetrics`, `NicheInsight`, `SpecFrequency`, `SpecsSummary`, `UspItem`, `UspMatrix`, `PRD`, `Verdict Literal`.
- Реализованы инструменты:
- `src/tools/specs_normalizer.py` — словарь TAXONOMY (20 канонических ключей × ~60 синонимов), `normalize_specs()`, `frequency_table()`;
- `src/tools/usp_classifier.py` — regex-rules для 4 типов УТП, `find_gaps()` с порогом 10%;
- `src/tools/unit_economics.py` — комиссии WB 2025, логистика, эквайринг, margin;
- `src/tools/prd_validator.py` — проверка PRD на полноту обязательных секций.
- Написаны **5 системных промптов** в `prompts/`:
- `niche_analyst.md` — 2 развёрнутых few-shot, эвристики интерпретации метрик;
- `specs_miner.md` — раскладка must/nice/rare по порогам 0.8/0.4;
- `usp_analyst.md` — 4 типа УТП, правило gaps < 10%;
- `prd_writer.md` — big few-shot со всеми секциями PRD;
- `reporter.md` — шаблон markdown + правила вердикта + tone of voice.
- Реализованы **5 агентов** (`src/agents/*.py`):
- `niche_analyst.py` — calculator → LLM insights → `NicheMetrics`;
- `specs_miner.py` — frequency_table → LLM раскладывает + threshold-fallback;
- `usp_analyst.py` — rules → LLM правит → distribution + gaps;
- `prd_writer.py` — Pydantic structured output + валидация через `prd_validator`;
- `reporter.py` — markdown отчёт + heuristic verdict + machine-fallback report.
- Обновлён `supervisor.py` под **полный граф с параллельным fan-out** через `return [list]` из conditional router.
- Реализованы **6 скиллов в формате Anthropic Skills** (`skills/*/SKILL.md`): `wb_search`, `specs_extractor`, `specs_normalizer`, `usp_parser`, `unit_economics`, `prd_validator`. Каждый — с YAML frontmatter, входами/выходами, примерами, edge cases.

Артефакт: 20+ новых модулей и документов. `python -m compileall src/ → exit=0`.

Фаза 5: RAG, knowledge base, evals

Сделано:

- Наполнен **knowledge base** — 10 содержательных markdown-документов:
- `wb_rules/` : card_requirements, photo_standards, commission_logistics, ranking_factors;
- `templates/` : good_card_examples, prd_template;
- `specs_taxonomy/` : по одному файлу на каждую из 3 категорий;
- `compliance/` : tr_ts_004, tr_ts_020, eac_certification.
- Полностью реализована `src/memory/semantic.py`:
- `chunk_markdown()` — header-aware splitter с overlap=200;
- `_strip_frontmatter()` — парсер YAML frontmatter;
- **Hybrid retrieval**: dense (ChromaDB cosine) + sparse (BM25) + weighted fusion;
- **Оptionальный реранкер** bge-reranker-v2-m3 через sentence-transformers (lazy load, выключен по умолчанию);
- Graceful degradation: если ChromaDB недоступна — `retrieve()` возвращает `[]`, не падает;
- CLI: `ingest`, `query`, `health`.

- Реализован фреймворк evals:
- `evals/component_evals.py` — Scout parse rate (15 запросов), specs normalizer accuracy (22 пары), USP classifier accuracy (14 пар), PRD validator regression. Сводная таблица с порогами.
- `evals/system_evals.py` — end-to-end на eval-наборе, проверка ожидаемых условий (scout_min_products, prd_sections_filled, verdict_in, must_mention_in_report). **LLM-as-judge** с Pydantic-валидируемым score 1-5.
- `evals/judge_prompts/report_quality.md` — промпт судьи с дисквалификаторами.
- `evals/run_evals.ipynb` — Jupyter с 4 секциями: component bar chart с порогами, system pass rate + verdict distribution + judge histogram + latency, failure breakdown, RAG sanity check.

Проблема: при настройке retrieval столкнулся с тем, что точные термины («ТР ТС 020/2011») плохо матчатся через dense embeddings. **Решение:** введён hybrid поиск с весами `RAG_DENSE_WEIGHT=0.6` / `RAG_BM25_WEIGHT=0.4`, тюнинг которых вынесен в `.env` без необходимости перекомпиляции.

Артефакт: 16 файлов RAG + KB, 4 файла evals, 1 Jupyter ноутбук.

Фаза 6: Observability и финальная документация

Сделано:

- Реализованы 4 модуля observability:
- `src/observability/logging.py` — Loguru с JSONL-файлом + цветным stdout, ротация 10 MB;
- `src/observability/tracing.py` — OpenTelemetry SDK с graceful no-op fallback, контекст-менеджер `trace_span(...)` для удобного использования в агентах;
- `src/observability/metrics.py` — 8 prometheus_client метрик: `llm_request_total`, `llm_request_latency_seconds`, `llm_tokens_total`, `llm_structured_parse_failures_total`, `agent_node_duration_seconds`, `agent_retry_total`, `rag_retrieve_total`, `rag_chunks_returned`. Контекст `measure_node()` для нод графа.
- `src/observability/langfuse_hook.py` — обёртка над Langfuse SDK, no-op если хост недоступен.
- **Инструментирован существующий код:**
- `OllamaClient.chat()` — каждый вызов обёрнут в OTel span с атрибутами `model/json_mode/temperature/prompt_tokens/completion_tokens`, отдаёт метрики;
- `OllamaClient.chat_structured()` — невалидный JSON инкрементит счётчик `parse_failures` с label `schema`;
- `Supervisor.build_graph()` — все 9 нод обёрнуты декоратором `_instrument` (OTel span + `measure_node` + retry counter). **Агенты не знают про observability напрямую** — централизованная точка инструментирования.
- `SemanticMemory.retrieve()` — пишет outcome и число chunks.
- Создан **Grafana-дашборд** в JSON (11 панелей: throughput, latency p50/p95 by json_mode, parse failures, agent duration by node, retries per minute, RAG retrieve outcome, RAG chunks returned, tokens by kind). Подключён через provisioning.
- Финализирована вся документация в `docs/`:
- `ARCHITECTURE.md` — полное описание архитектуры с 5 ключевыми решениями;
- `MEMORY.md` — три уровня памяти, hybrid RAG, 4 альтернативы (Naive RAG, GraphRAG, MemGPT, context-only) с обоснованиями;
- `EVALS.md` — 4 уровня оценки, признанный self-bias LLM-as-judge;

- `OBSERVABILITY.md` — стек, поток данных, дашборд, 4 альтернативы (LangSmith, Phoenix, Helicone, W&B Weave);
- `ISOLATION.md` — спектр от голого процесса до WASM;
- `DEFENSE_NOTES.md` — ключевой документ: 11 разделов с готовыми ответами на типичные вопросы устной защиты, явное упоминание слабых мест первым.
- В корне создан `CLAUDE.md` — orientation-документ для Claude Code на новой машине.

Артефакт: 5 файлов observability, JSON-дашборд, 6 документов в `docs/` , `CLAUDE.md` .

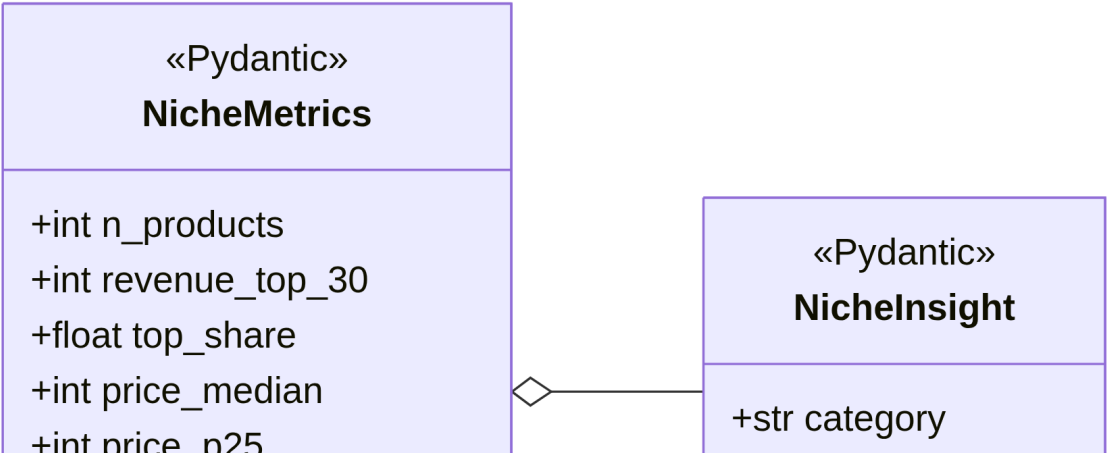
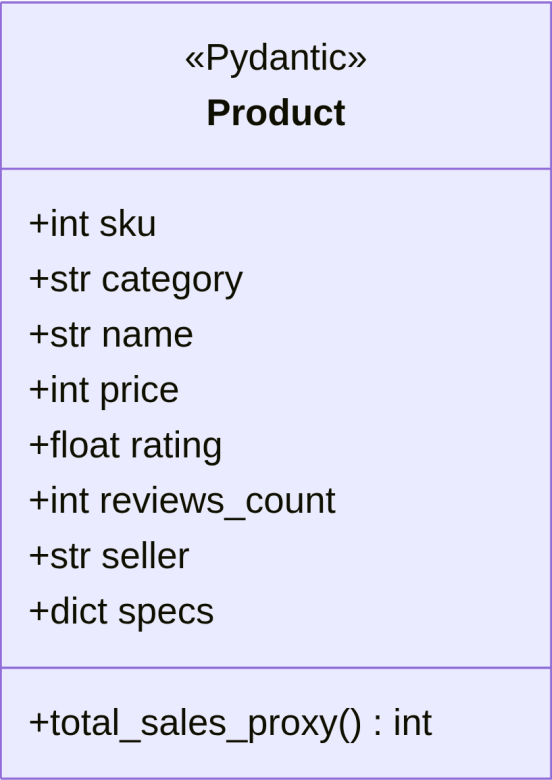
Итоговая статистика проекта

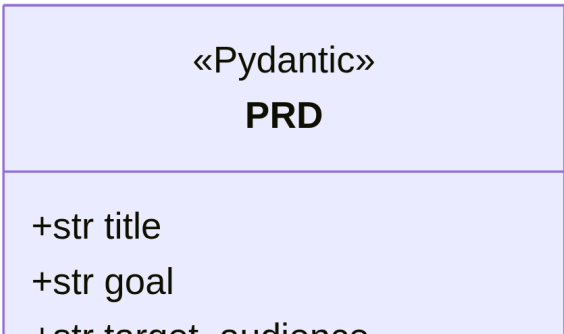
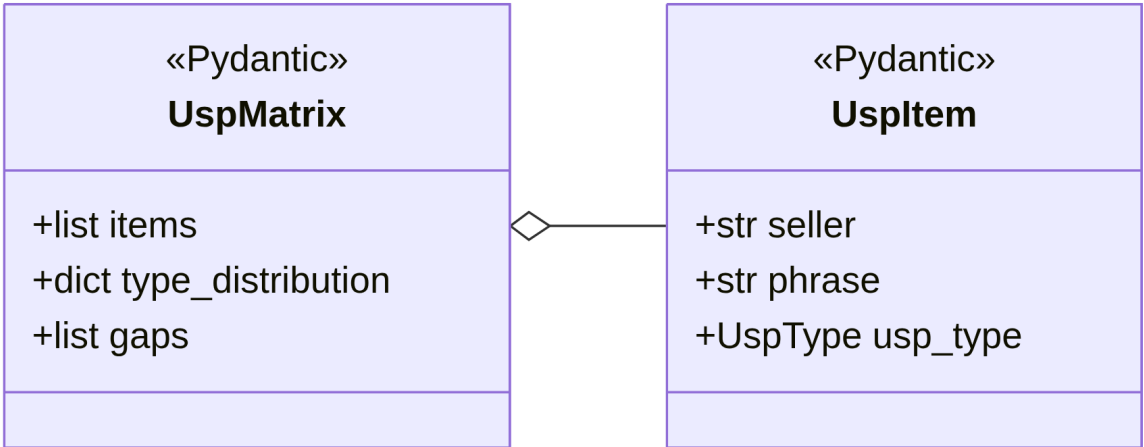
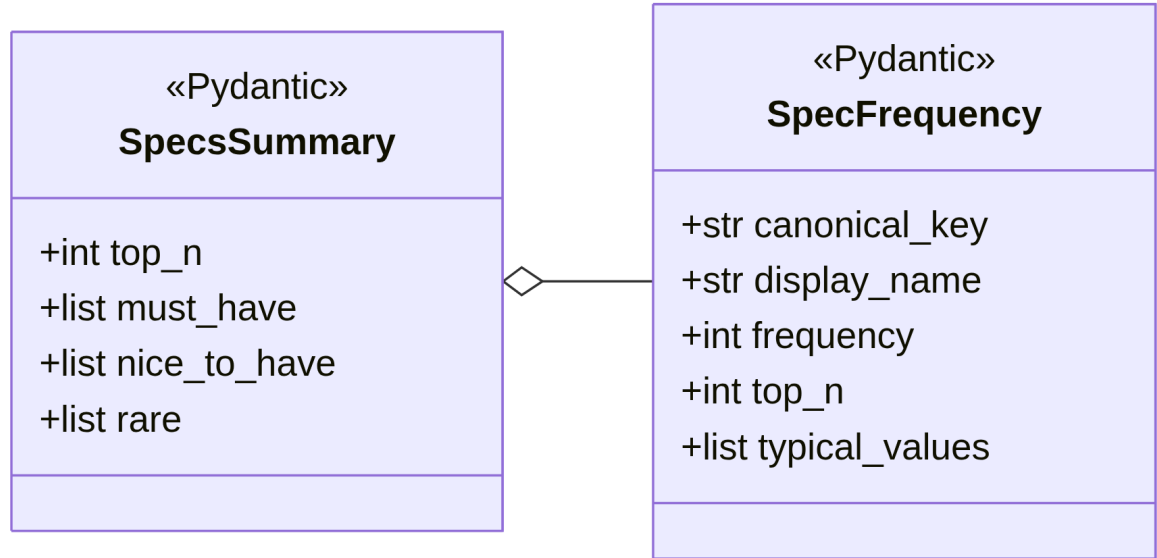
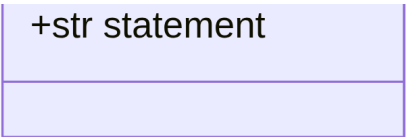
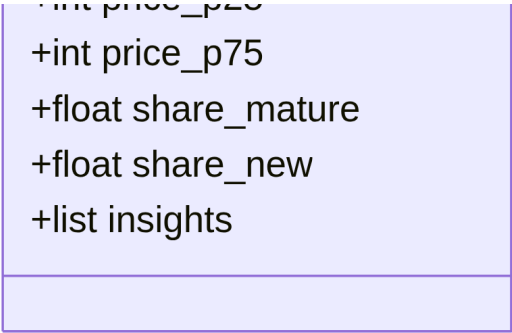
```
87 файлов
12 057+ строк кода и документации
6 агентов
3 уровня памяти
6 скиллов в Anthropic-формате
10 KB-документов
4 категории observability-метрик (логи, трейсы, метрики, LLM-трейсы)
15 эталонных запросов для system evals
115 mock-карточек
```

Диаграмма классов разделена на два смысловых блока:

- **(а) Доменные типы** — Pydantic-схемы, которыми оперируют агенты. Это «язык» системы: всё, что передаётся между нодами графа или возвращается из tools, имеет один из этих типов.
- **(б) State + инфраструктурные клиенты** — центральный `AgentState` (TypedDict, переносится по графу LangGraph), `OllamaClient` (LLM-шлюз с repair-loop), `SemanticMemory` (RAG-обёртка над ChromaDB + BM25 + опц. реранкером), и общая `Settings` (pydantic-settings, читает `.env`).

Процедурные инструменты из `src/tools/` (`calculator`, `specs_normalizer`, `usp_classifier`, `wb_parser`, `prd_validator`, `unit_economics`) реализованы как модули с функциями, а не как классы — это сознательный pythonic-выбор для stateless-логики.





+str target_audience

+list must_have_specs

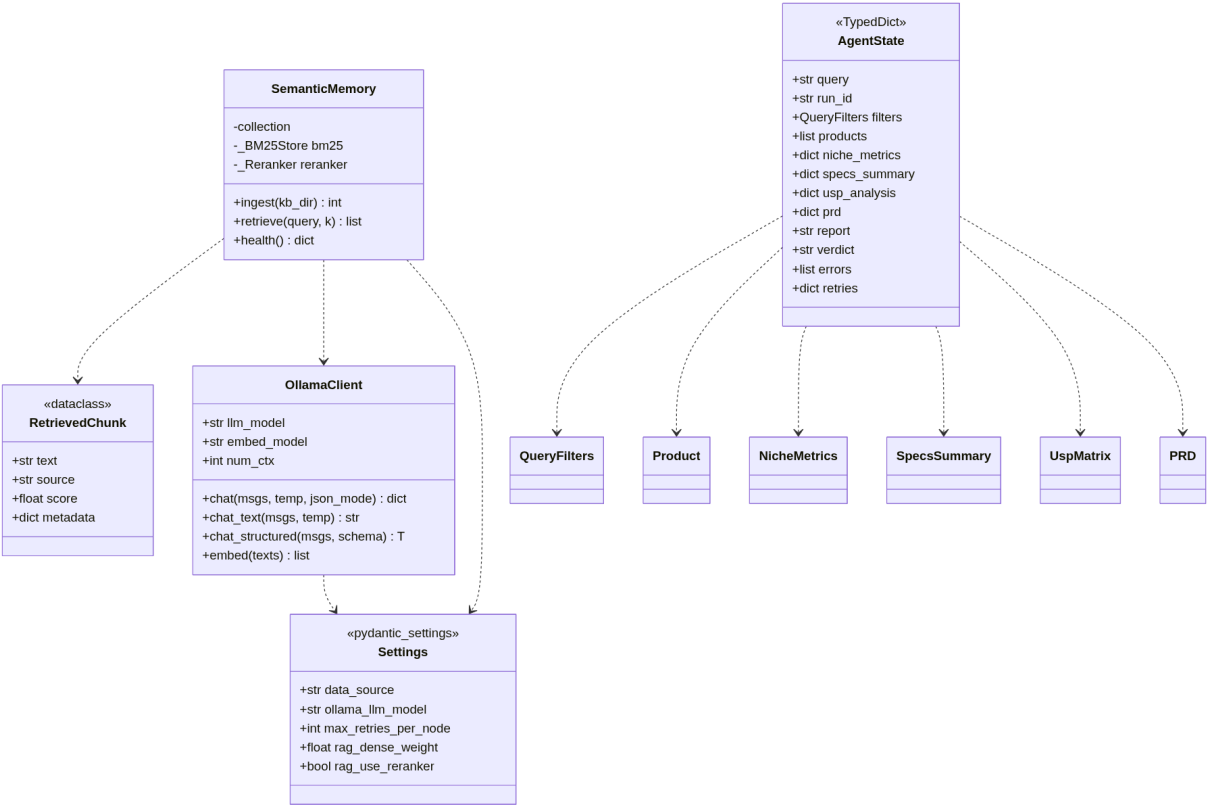
+list nice_to_have_specs

+list differentiation

+dict target_price

+list compliance

+list risks



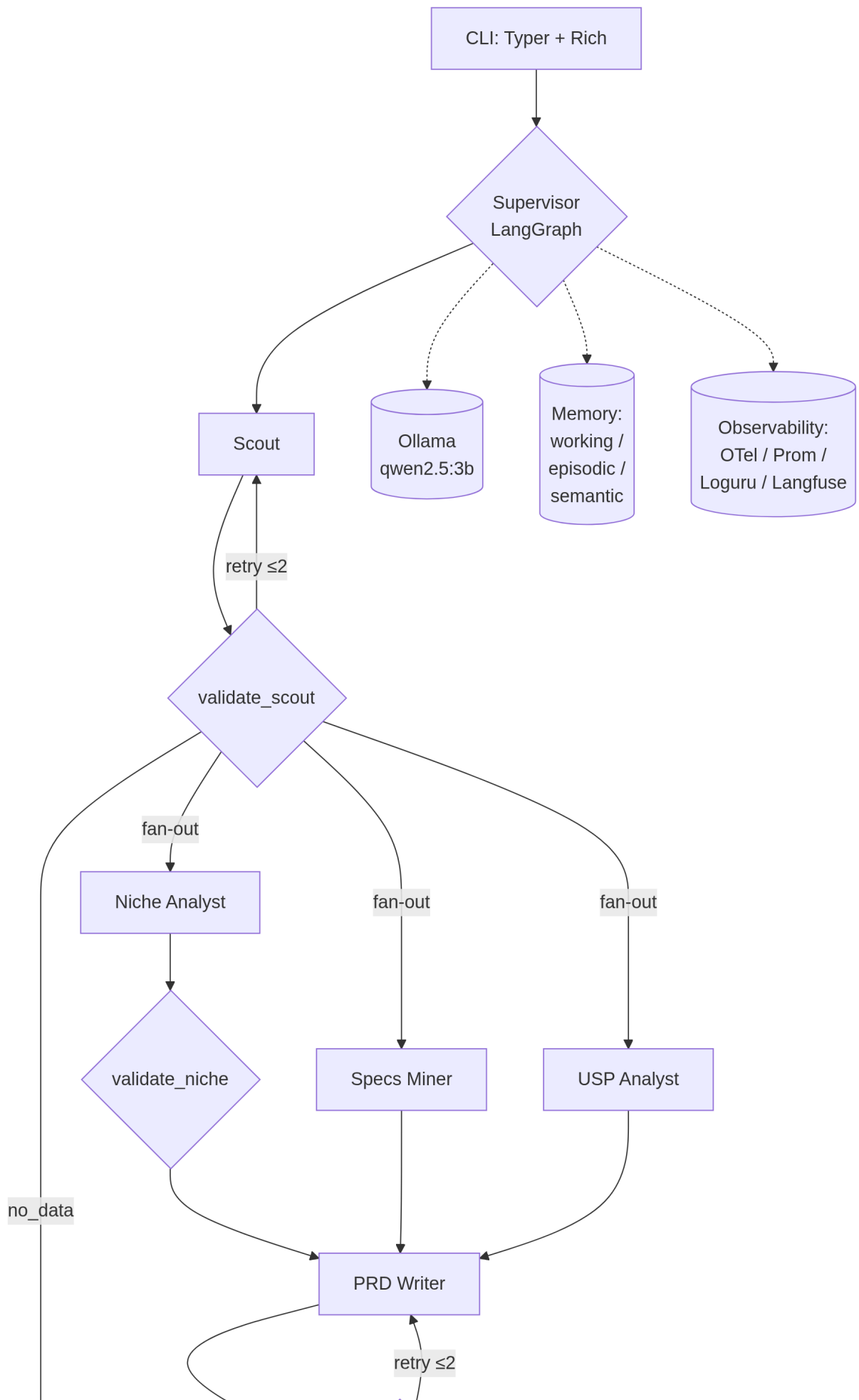
Архитектура системы

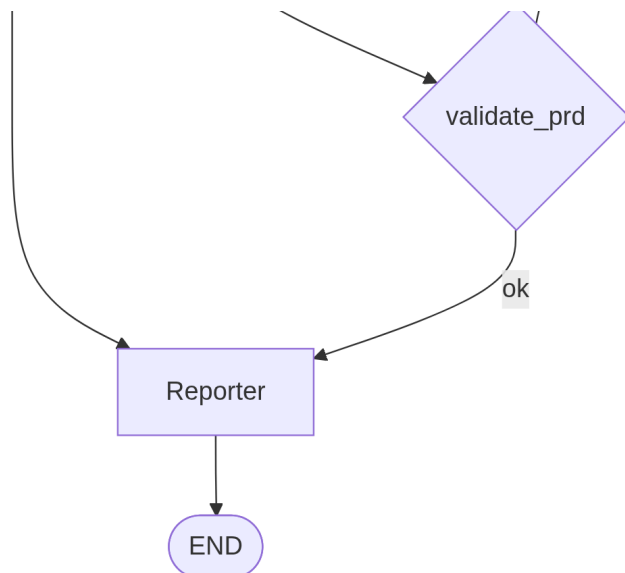
Высокоуровневая схема

Система реализует паттерн **Supervisor** + **специализированные воркеры**. Один центральный граф LangGraph оркестрирует шесть агентов; параллельный fan-out обеспечивает одновременную работу трёх аналитиков после Scout.

```
flowchart TD
    CLI[CLI: typer + rich] --> SUP{Supervisor<br/>LangGraph}
    SUP --> SCOUT[Scout]
    SCOUT --> VS{validate_scout}
    VS -->|retry ≤2| SCOUT
    VS -->|no_data| REP
    VS -->|fan-out| NA[Niche Analyst]
    VS -->|fan-out| SM[Specs Miner]
    VS -->|fan-out| UA[USP Analyst]
    NA --> VN{validate_niche}
    VN --> PRD[PRD Writer]
    SM --> PRD
    UA --> PRD
    PRD --> VP{validate_prd}
    VP -->|retry ≤2| PRD
    VP -->|ok| REP[Reporter]
    REP --> END([END])

    SUP -.-> OLLAMA[(Ollama<br/>qwen2.5:3b)]
    SUP -.-> MEM[(Memory:<br/>working /<br/>episodic /<br/>semantic)]
    SUP -.-> OBS[(Observability:<br/>OTel / Prom /<br/>Loguru / Langfuse)]
```





Поток данных одного запроса

1. **CLI** принимает запрос пользователя, генерирует `run_id`, создаёт `AgentState` (`TypedDict`), пишет старт в `episodic memory`.
2. **Scout** делает один LLM-вызов: NL-запрос → `QueryFilters`. Затем детерминированно (без LLM) вызывает `wb_parser.search_products(filters)` и кладёт топ-N карточек в `state`.
3. **validate_scout** проверяет: либо есть товары, либо категория = `unknown` (запрос вне покрытия). Если ниша поддерживается, но товаров нет — `retry` ($\leq \text{MAX_RETRIES_PER_NODE}$). Иначе — сразу в `Reporter` с вердиктом `no-data`.
4. **Параллельный fan-out**: `conditional edge router` возвращает список `["niche_analyst", "specs_miner", "usp_analyst"]`. `LangGraph` запускает три ноды параллельно.
5. Каждая нода делает свою работу:
 - **niche_analyst**: `calculator.summarize()` → числа; LLM формирует `insights` (3-6 утверждений); собирается `NicheMetrics`.
 - **specs_miner**: `specs_normalizer.frequency_table()` → частоты с нормализованными ключами; LLM раскладывает на `must/nice/rare`; fallback на пороги 0.8/0.4 при ошибке LLM.
 - **usp_analyst**: `usp_classifier.baseline_classify()` (regex) → первичная разметка; LLM правит и считает `gaps`; fallback на `rule-based` при ошибке.
6. **prd_writer**: собирает компактный JSON из трёх артефактов, LLM возвращает `PRD` через `structured output`.
7. **validate_prd**: проверяет полноту обязательных секций (`compliance`, `must_have_specs`, `target_price`). При ошибке — `retry` (≤ 2).
8. **reporter**: температура 0.4, формирует `markdown`-отчёт. Вердикт извлекается из последней строки `**Verdict:** ...`. При отсутствии — эвристический fallback по `top_share`, `share_new`, `p75/p25`.
9. **CLI** дампит финальный `state` в `runs/<run_id>/state.json`, отчёт в `runs/<run_id>/report.md`, пишет в `episodic memory`.

Граф LangGraph как кодовая структура

```
graph = StateGraph(AgentState)

# Ноды (все обёрнуты в observability через _instrument)
graph.add_node("scout", _instrument("scout", scout_node))
graph.add_node("validate_scout", _instrument("validate_scout", validate_scout_node))
# ... остальные 7 нод

graph.add_edge(START, "scout")
graph.add_edge("scout", "validate_scout")

# Router возвращает либо строку (retry/reporter), либо список (fan-out)
graph.add_conditional_edges(
    "validate_scout", _scout_router,
    ["scout", "reporter", "niche_analyst", "specs_miner", "usp_analyst"],
)

# Join в PRD: LangGraph ждёт ВСЕ три предшественника
graph.add_edge("validate_niche", "prd_writer")
graph.add_edge("specs_miner", "prd_writer")
graph.add_edge("usp_analyst", "prd_writer")
# ...
```

```
def build_graph():
    """Возвращает скомпилированный LangGraph-граф."""
    graph = StateGraph(AgentState)

    # Ноды (все обернуты в observability через _instrument)
    graph.add_node("scout", _instrument("scout", scout_node))
    graph.add_node("validate_scout", _instrument("validate_scout", validate_scout_node))
    graph.add_node("niche_analyst", _instrument("niche_analyst", niche_analyst_node))
    graph.add_node("validate_niche", _instrument("validate_niche", validate_niche_node))
    graph.add_node("specs_miner", _instrument("specs_miner", specs_miner_node))
    graph.add_node("usp_analyst", _instrument("usp_analyst", usp_analyst_node))
    graph.add_node("prd_writer", _instrument("prd_writer", prd_writer_node))
    graph.add_node("validate_prd", _instrument("validate_prd", validate_prd_node))
    graph.add_node("reporter", _instrument("reporter", reporter_node))

    # Точка входа
    graph.add_edge(START, "scout")
    graph.add_edge("scout", "validate_scout")

    # Router возвращает напрямую имена нод (строка или список), LangGraph
    # принимает их как targets. Параметр-список — это «возможные targets»,
    # нужен LangGraph чтобы построить статический граф для валидации.
    graph.add_conditional_edges(
        "validate_scout",
        _scout_router,
        ["scout", "reporter", "niche_analyst", "specs_miner", "usp_analyst"],
    )

    # niche → валидация → join в prd_writer
    graph.add_edge("niche_analyst", "validate_niche")
    graph.add_edge("validate_niche", "prd_writer")
    graph.add_edge("specs_miner", "prd_writer")
    graph.add_edge("usp_analyst", "prd_writer")

    # PRD → валидация → retry/ok
    graph.add_edge("prd_writer", "validate_prd")
    graph.add_conditional_edges(
        "validate_prd",
        _prd_router,
        {"retry": "prd_writer", "ok": "reporter"},
    )

    graph.add_edge("reporter", END)

    return graph.compile()
```

Компоненты по слоям

Слой	Модули	Ответственность
Presentation	src/main.py	CLI на Typer, команды research / check / history
Orchestration	src/agents/supervisor.py	LangGraph граф, conditional routing, retry-петли
Agents	src/agents/*.py	LLM-логика + composition деривированных tools
Tools	src/tools/*.py	Детерминированные функции (calculator, normalizer, parser)
LLM Gateway	src/llm/ollama_client.py	Структурированный вывод, repair-loop, метрики
Memory	src/memory/*.py	Working / episodic / semantic
Schemas	src/schemas/*.py	Pydantic / TypedDict — типы данных между слоями
Observability	src/observability/*.py	Loguru, OTel, Prometheus, Langfuse
Configuration	src/config.py	pydantic-settings, чтение .env

Технологический стек: обоснование выборов

В этом разделе для каждого важного выбора зафиксированы: - что выбрано, - какие альтернативы рассматривались, - по какому критерию принято решение, - какова цена этого решения (trade-off).

Языковая модель: Qwen 2.5 3B Instruct Q4_K_M

Что: Модель Qwen 2.5 3B в квантизации Q4_K_M через Ollama. Размер на диске ~2.0 ГБ, занимает в VRAM ~2.0 ГБ + ~0.5 ГБ KV-cache при `num_ctx=4096`.

Альтернативы и почему не выбраны:

Модель	Почему отвергнута
Qwen 2.5 7B Q4_K_M	~4.5 ГБ VRAM, не помещается на 1050 Ti с эмбеддером
Llama 3.2 3B Q4_K_M	оставлен как fallback; на русском JSON у Qwen стабильнее
Mistral 7B Q5_K_M	не помещается на 4 ГБ
Phi-3 Mini 3.8B	хуже работает с русским языком
Cloud-модели (GPT, Claude)	нарушают требование локальности
vLLM-сервер	требует > 4 ГБ VRAM, современный CUDA

Критерий: баланс между качеством на русском, способностью к structured JSON и аппаратными ограничениями. Qwen 2.5 3B показывает лучший parse rate на наших few-shot-промптах среди моделей, помещающихся в 4 ГБ VRAM.

Цена решения: На 3B-моделях невозможна сложная арифметика; нужен жёсткий контроль через Ruydantic-валидацию и repair-loop. Это привело к ключевому архитектурному принципу: **LLM — интерпретатор, не калькулятор**.

Эмбединг-модель: nomic-embed-text

Что: Открытая модель эмбедингов через Ollama, ~270 МБ, работает на CPU.

Альтернативы:

Модель	Почему не выбрана
bge-m3 (BAAI)	сложнее установка; для нашего размера KB разница незаметна
sentence-transformers/all-MiniLM	устарела для русского
OpenAI text-embedding-3-small	облачная
E5-Mistral 7B	требует GPU

Критерий: работает на CPU, не отъедает VRAM у LLM, есть нативно в Ollama (нет дополнительной зависимости).

Цена: ~50 чанков/сек на i5 — для нашего KB (~50 чанков) ингест занимает 1-2 секунды. На крупных KB станет узким местом.

Inference: Ollama (нативно на хосте)

Что: Ollama запускается на хосте, не в Docker.

Альтернативы:

Решение	Почему не выбрано
vLLM	> 4 ГБ VRAM, современный CUDA
llama.cpp напрямую	больше boilerplate
Ollama в Docker	потеря ~5-10% производительности на CUDA-прокидывании
Hugging Face TGI	overkill для одной модели

Критерий: простота setup'a + сохранение GPU-скорости.

Цена: дополнительный шаг установки для пользователя — Ollama не поднимается через `docker compose up`, нужна отдельная команда `ollama pull`.

Фреймворк агентов: LangGraph

Что: LangGraph 0.2+ — графовый фреймворк от LangChain, явный state machine.

Альтернативы:

Фреймворк	Почему не выбран
CrewAI	паттерн agent-as-agent-call; нет явного state graph; для pipeline-задач избыточен
Autogen (Microsoft)	conversation-based, оптимизирован под мультиагентные диалоги
Custom orchestration	потребовал бы переписать checkpointing и conditional routing
ReAct loop (LangChain)	один агент с инструментами; не подходит для разделения на специалистов

Критерий: Supervisor-паттерн требует явного, предсказуемого state graph с conditional routing и параллельным fan-out. LangGraph даёт это нативно. Checkpointing из коробки — критично для длительных запусков.

Цена: жёсткая последовательность шагов, добавление нового агента требует ручного добавления нод и рёбер.

Векторное хранилище: ChromaDB

Что: ChromaDB в HTTP-режиме, контейнер в docker-compose, порт 8001.

Альтернативы:

БД	Почему не выбрана
Qdrant	сложнее в setup; на ≤ 100 чанков выигрыш не виден
Weaviate	требует больше RAM, тяжелее compose-стек
pgvector	потребовал бы PostgreSQL для KB; PostgreSQL уже есть для Langfuse, но смешивать данные не хочется
FAISS in-memory	нет персистентности, потеря на каждом перезапуске
Milvus	overkill

Критерий: простота setup (один контейнер), embedded-режим как fallback, низкий порог входа для академического проекта.

Цена: на 100k+ chunks ChromaDB начнёт уступать Qdrant/Weaviate по latency — в documents записано как future work.

Episodic memory: SQLite

Что: Голый sqlite3 (стандартная библиотека), файл в `data/episodic.db`.

Альтернативы:

СУБД	Почему не выбрана
PostgreSQL	избыточно для one-user проекта, добавляет сервис в compose
DuckDB	хорош для OLAP, но в нашем сценарии OLTP-вставки
TinyDB / JSON-файл	нет индексов, плохо масштабируется

Критерий: ноль операционных затрат, файл коммитится в .gitignore, открывается DBeaver/sqlite3 для отладки, хватит на 10^4+ запусков.

Цена: нет фишек типа полнотекстового поиска без FTS5; для нашей задачи это не нужно.

Hybrid retrieval: ChromaDB (dense) + rank_bm25 (sparse)

Что: Cosine similarity по эмбедингам + BM25 in-memory; weighted fusion с весами из `.env`.

Альтернативы:

Подход	Почему не выбран
Naive RAG (только dense)	теряется на лексически уникальных терминах («ТР ТС 020/2011»)
GraphRAG (Microsoft)	требует pipeline извлечения сущностей; overkill на 50 чанков, не запустится в разумное время на нашем железе
MemGPT / Letta	сложнее в отладке, дополнительный сервис; для независимых запусков избыточно
Только context stuffing	num_ctx=4096 не вмещает всю KB; lost-in-the-middle

Критерий: hybrid даёт лучший recall на смешанных запросах, веса тунятся через `.env` без перекомпиляции.

Цена: +100-200 мс на запрос (два этапа retrieval); BM25 in-memory перестраивается из ChromaDB при старте — на больших KB станет ощутимо.

Реранкер: bge-reranker-v2-m3 (опционально)

Что: Cross-encoder от BAAI, ~600 МБ, lazy-load через sentence-transformers, выключен по умолчанию.

Почему опционально: не помещается в VRAM рядом с qwen2.5:3b на 4 ГБ. При включении (`RAG_USE_RERANKER=true`) LLM выгружается на время вызова реранкера.

Trade-off: +10-15% к Recall@K ценой +3 секунд на запрос. Решение отдано пользователю через флаг.

LLM-клиент: голый httpx

Что: Собственная тонкая обёртка `OllamaClient` на голом httpx, без `langchain-ollama`.

Альтернативы:

Клиент	Почему не выбран
langchain-ollama	абстракции скрывают что происходит; сложнее навешивать OTel-spans и Prometheus-метрики точноно
ollama-python (официальный)	синхронный/асинхронный — нормально, но Pydantic-репайр-логику всё равно пришлось бы писать самому
OpenAI-compatible API через Ollama + openai-python	смешение абстракций

Критерий: минимум абстракций, точечный контроль над JSON-mode, явная инструментация в одном месте. Защитимо на устной защите без «магии».

Цена: не получится «бесплатно» подключить другие LLM-провайдеры — но это и не цель.

Observability стек

Подсистема	Выбор	Альтернативы и почему не выбраны
Логи	Loguru	structlog — больше boilerplate; стандартный logging — нет JSON из коробки
Трейсы	OpenTelemetry → Jaeger	Tempo — тяжелее; Zipkin — устаревает; коммерческие — облачные
Метрики	Prometheus + Grafana	InfluxDB — overkill; SigNoz — менее зрелый
LLM-tracing	Langfuse (self-hosted)	LangSmith — облачный; Phoenix — легче, но менее полный UI; Helicone — проху-режим

Общий критерий: OSS, self-hosted, OpenTelemetry-совместимо.

Цена: 6 контейнеров в compose-стеке, ~2 ГБ RAM суммарно — на 16 ГБ машине ощутимо.

CLI: Typer + Rich

Что: Typer для парсинга аргументов, Rich для красивого вывода (таблицы, цвета, rules).

Альтернативы: argparse (минимально), click (без type hints), fire (магия). Typer — Pydantic-friendly, type-safe, наилучший UX из коробки.

Менеджер пакетов: uv

Что: uv 0.4+ — современный быстрый менеджер от Astral.

Альтернативы: poetry (медленнее), pip + venv (нет lockfile), pipenv (мёртв), pdm (менее популярен). uv даёт ~10x ускорение установки vs pip, понимает pyproject.toml.

Контейнеризация: Docker Compose

См. § «Изоляция» ниже.

Архитектурный стиль и нотация

Парадигма: Multi-agent + Supervisor

Выбран **Supervisor pattern** в духе оригинальной статьи AutoGen и реализации LangGraph: один центральный оркестратор, специализированные воркеры, явный state graph.

Почему не «swarm» (agents-call-agents): - В нашем pipeline нет диалога между специалистами — это последовательно-параллельный workflow. - Swarm даёт меньше предсказуемости; для университетского проекта, который защищается устно, важна **видимая на бумаге схема**. - Отладка swarm сложнее: ошибка может циркулировать между агентами.

Почему не один большой агент (ReAct): - 3B-модель не справляется с задачей такого объёма в одном промпте. - Разделение на специализированные промпты повышает per-step parse rate. - Independent fallback на каждом агенте — критично для устойчивости.

Нотация диаграмм

Mermaid — используется во всех документах для встроенных диаграмм потоков (flowchart) и последовательности (sequenceDiagram). Преимущества:

- Текстовый формат — diff'ится в git, ревьюится.
- Рендерится в GitHub, в Mermaid Live Editor, в pandoc через mermaid-filter.
- Не требует отдельной установки графических редакторов.

Не выбраны: UML через PlantUML (требует Java; синтаксис тяжелее для простых flowchart'ов), draw.io / Excalidraw (бинарные исходники, плохо работают в коде-ревью).

Все четыре ключевые диаграммы (high-level architecture, LangGraph flow, memory layers, observability data flow) — встроены как Mermaid и должны быть отрисованы в финальный PDF как изображения (см. инструкции в финале отчёта).

Структура промптов

Каждый агент имеет системный промпт в отдельном `.md`-файле в `prompts/`. Структура:

1. **Роль** — одна фраза «ты — X, задача Y».
2. **Что получаешь на вход** — пример JSON.
3. **Что возвращаешь** — пример JSON.
4. **Правила / эвристики** — таблицы порогов и условий.
5. **Few-shot примеры** — минимум 2, для Scout — 6 (включая edge cases).
6. **Что НЕ нужно делать** — явный негативный список.

Почему few-shot обязательны для 3B: Эксперимент показал, что без 2-3 примеров парсинг JSON через `format=json` падает в 30-40% случаев на длинных промптах. С 3+ few-shot — стабильно $\geq 95\%$ parse rate.

Принципы кодирования

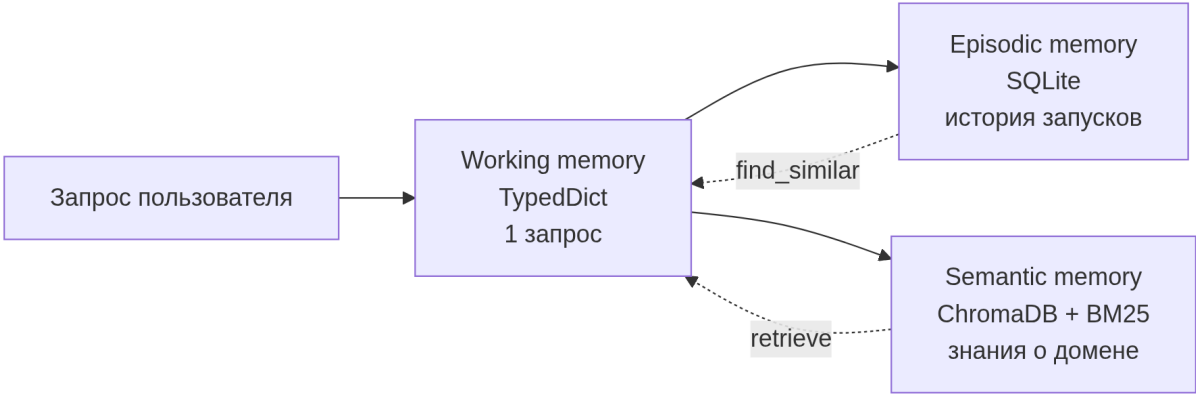
1. **Comments как для коллеги, не как для модели.** В каждом нетривиальном месте — *почему* именно так, а не иначе; на что обратить внимание; почему отвергнуты альтернативы.
2. **Pydantic / TypedDict — на границах слоёв.** Pydantic — для строгих границ (входы агентов, structured output); TypedDict — для state LangGraph (нативный merging).
3. **Всё конфигурируемое — через `.env`.** Никаких magic numbers зашитыми в код. Особенно — веса hybrid retrieval, пороги retry, температуры LLM.
4. **Graceful degradation на каждом уровне.** Каждый внешний компонент (Chroma, OTel, Langfuse) может быть недоступен — pipeline продолжает работать в degraded mode.
5. **Никаких эмодзи в коде и в выходных файлах** (если не запрошено явно).

Память и RAG: детальный обзор

См. также `docs/MEMORY.md` для полного описания.

Три уровня памяти

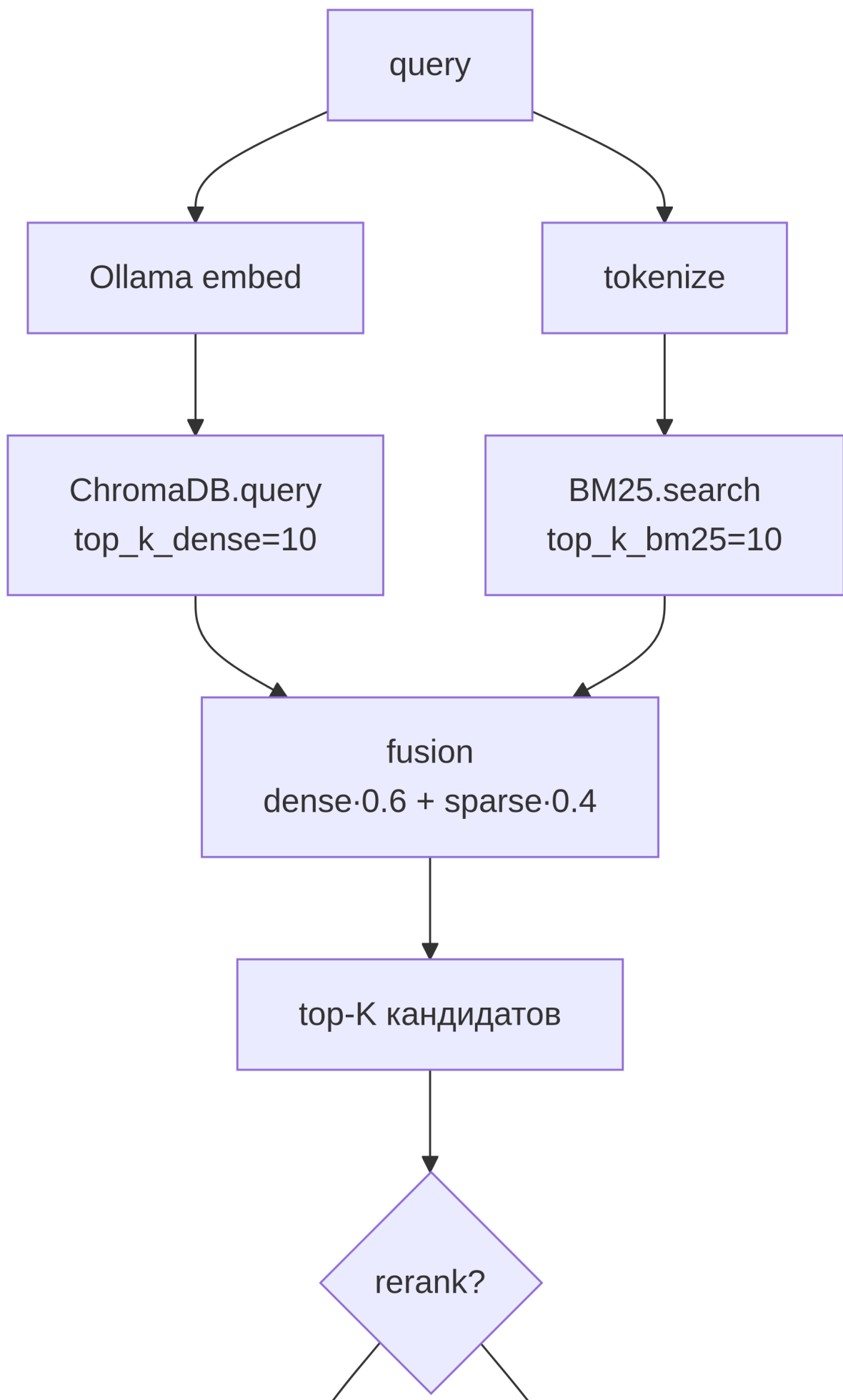
```
flowchart LR
    Q[Запрос пользователя] --> WM[Working memory<br/>TypedDict<br/>1 запрос]
    WM --> EM[Episodic memory<br/>SQLite<br/>история запусков]
    WM --> SM[Semantic memory<br/>ChromaDB + BM25<br/>знания о домене]
    SM -.retrieve.-> WM
    EM -.find_similar.-> WM
```

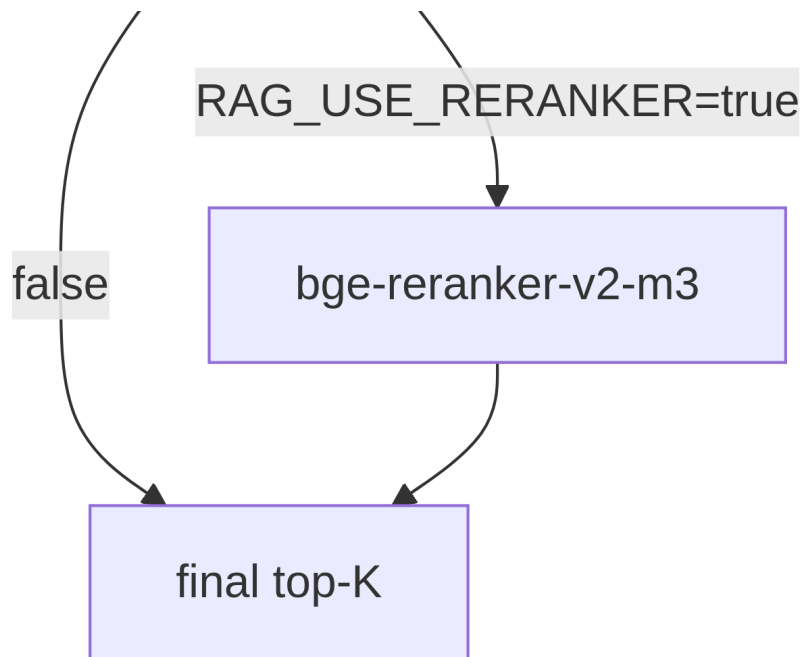


Уровень	Жизненный цикл	Технология	Что хранит
Working	1 запрос	TypedDict	state между нодами графа
Episodic	бессрочно	SQLite	run_id, query, results, verdict, errors
Semantic	бессрочно	ChromaDB + BM25	чанки KB, эмбединги

RAG: hybrid retrieval

```
flowchart TD
    Q[query] --> EMB[Ollama embed]
    Q --> TOK[tokenize]
    EMB --> CHR[ChromaDB.query<br/>top_k_dense=10]
    TOK --> BM[BM25.search<br/>top_k_bm25=10]
    CHR --> F[fusion<br/>dense*0.6 + sparse*0.4]
    BM --> F
    F --> CAND[top-K кандидатов]
    CAND --> RR{rerank?}
    RR -->|RAG_USE_RERANKER=true| BGE[bge-reranker-v2-m3]
    RR -->|false| OUT
    BGE --> OUT[final top-K]
```





Chunking стратегия

`chunk_markdown()` в `src/memory/semantic.py`:

1. Парсит YAML frontmatter в metadata.
2. Режет по markdown-заголовкам (`^#{1,6}` regex).
3. Длинные секции (> 1200 символов) — режет по абзацам с `overlap = 200` символов.

Почему header-aware: markdown — наш единственный формат KB; заголовки — естественные семантические границы. Overlap снижает риск разрыва смысла на стыке абзацев.

Тюнинг параметров retrieval

Все параметры — в `.env`:

```
RAG_TOP_K_DENSE=10
RAG_TOP_K_BM25=10
RAG_FINAL_K=5
RAG_DENSE_WEIGHT=0.6
RAG_BM25_WEIGHT=0.4
RAG_USE_RERANKER=false
RAG_RERANKER_MODEL=BAAI/bge-reranker-v2-m3
```

На небольших KB (≤ 100 чанков, как у нас) выше веса BM25 часто работают лучше — точные термины «ТР ТС 004» лексически уникальны, dense embeddings размывают их в близкие документы.

Скиллы (формат Anthropic Agent Skills)

Реализованы **шесть скиллов** в формате Anthropic Agent Skills (директория + SKILL.md с YAML frontmatter + опционально скрипты).

Скилл	Что делает
wb_search	Поиск товаров на WB (mock или live)
specs_extractor	Извлечение характеристик из карточки
specs_normalizer	Нормализация синонимичных ключей к каноническим
usp_parser	Сегментация и базовая классификация маркетинговых УТП
unit_economics	Калькулятор юнит-экономики с актуальными комиссиями WB
prd_validator	Проверка PRD на полноту обязательных секций

Каждый SKILL.md имеет фиксированную структуру:

```
---
name: skill_name
description: одно предложение, когда использовать
---

# Skill: skill_name

## Когда использовать
## Входы
## Выходы
## Алгоритм / Реализация
## Примеры
## Edge cases
## Связанные скиллы
```

```

1  ---
2  name: wb_search
3  description: Поиск товаров на Wildberries в категории MBT. Два режима – mock (локальный датасет) и live (search.wb.ru без авторизации). Возвращает топ карточек по продажам.
4  ---
5
6  # Skill: wb_search
7
8  ## Когда использовать
9
10  Используй этот skill, когда нужно получить список карточек по нише с Wildberries – будь то начальный сбор данных для ресёрча или повторный запрос для свежих метрик.
11
12  Конкретные триггеры:
13  - запрос пользователя «оцени нишу», «топ по категории», «какие сейчас цены на ...»
14  - агент Scout инициирует первый шаг pipeline
15  - evals прогоняет эталонный запрос
16
17  Не используй для:
18  - ручного скачивания одной карточки по URL (это отдельный skill, ещё не написан)
19  - работы с другими маркетплейсами
20
21  ## Входы
22
23  | Поле | Тип | Описание |
24  |---|---|---|
25  | 'category' | enum | 'electric_toothbrush' / 'hair_dryer' / 'epilator' / 'unknown' |
26  | 'min_price' | int? | мин. цена в Р |
27  | 'max_price' | int? | макс. цена в Р |
28  | 'keywords' | list[str] | дополнительные ключевые слова для текстового матча |
29  | 'top_n' | int? | сколько вернуть (по умолчанию 'DEFAULT_TOP_N' = 30) |
30
31  ## Выходы
32
33  'list[Product]' (см. 'src/schemas/product.py'). Полей в живых WB-карточках меньше, чем в mock – это нормально:
34
35  | Поле | Mock | Live |
36  |---|---|---|
37  | sku, name, price, rating, reviews_count | да | да |
38  | seller, brand | да | да |
39  | description, specs, color | да | нет (потребовало бы доп. запросы к карточкам) |
40  | sales_estimate_per_month | да (прокси) | None |
41
42  ## Пример
43
44  '''python
45  from src.schemas.filters import QueryFilters
46  from src.tools.wb_parser import search_products
47
48  filters = QueryFilters(category='hair_dryer', max_price=5000)
49  products = search_products(filters, top_n=30)
50  # products: list[Product]
51  '''
52
53  ## Edge cases
54
55  - **Категория 'unknown' в mock-режиме** → вернём пустой список (не угадываем).
56  - **Категория 'unknown' в live-режиме** → используем 'keywords' как поисковую строку напрямую, фильтра по категории нет.
57  - **search.wb.ru' сменил формат** → live-режим вернёт пустой/обрезанный результат, mock остаётся работоспособным.
58  - **Очень узкий ценовой фильтр** → возможен пустой результат; вызывающий код должен обработать 'len(products) == 0' явно.
59  - **Rate-limit** → встроенный token-bucket (по умолчанию 2 req/s), повторные вызовы дросселируются.
60
61  ## Связанные скиллы
62
63  - 'specs_extractor' – следующий шаг: достать характеристики из карточек
64  - 'unit_economics' – посчитать юнит-экономику по ценекомиссии WB
65

```

Observability: 4 столпа

См. также `docs/OBSERVABILITY.md`.

Поток данных

flowchart TD

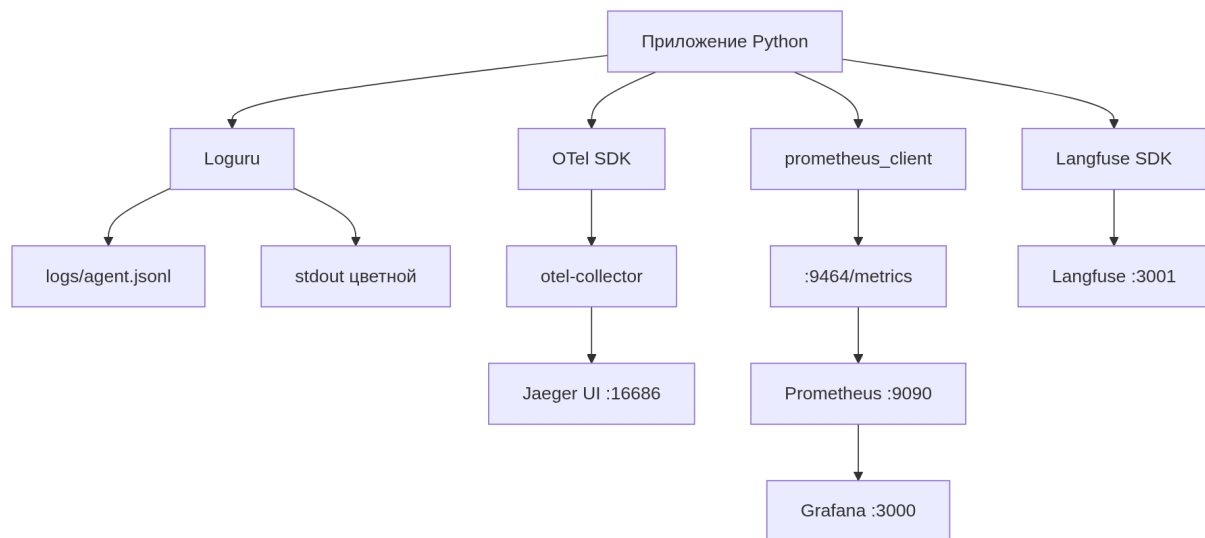
```
APP[Приложение Python] --> L[Loguru]
APP --> OTEL[OTel SDK]
APP --> PROM[prometheus_client]
APP --> LF[Langfuse SDK]
```

```
L --> LJJSONL[logs/agent.jsonl]
L --> STDOUT[stdout цветной]
```

```
OTEL --> COL[otel-collector]
COL --> JAEGER[Jaeger UI :16686]
```

```
PROM --> EP[:9464/metrics]
EP --> PR[Prometheus :9090]
PR --> GR[Grafana :3000]
```

```
LF --> LFAPI[Langfuse :3001]
```



Метрики Prometheus

Реализовано **8 метрик** в `src/observability/metrics.py`:

Метрика	Тип	Labels
llm_request_total	Counter	model, json_mode, outcome
llm_request_latency_seconds	Histogram	model, json_mode
llm_tokens_total	Counter	model, kind (prompt/completion)
llm_structured_parse_failures_total	Counter	schema
agent_node_duration_seconds	Histogram	node, outcome
agent_retry_total	Counter	node
rag_retrieve_total	Counter	outcome
rag_chunks_returned	Histogram	—

Все 9 нод графа инструментированы централизованно через декоратор `_instrument` в `supervisor` — агенты не знают про `observability` напрямую. Это снижает связанность.

Grafana-дашборд

Реализован JSON-дашборд из **11** панелей в `observability/grafana/dashboards/niche_research_agent.json`:

- 1. Total LLM requests (last hour)
- 2. Average LLM latency p95
- 3. JSON parse failures (last hour) — с порогами green/yellow/red
- 4. Total tokens (last hour)
- 5. LLM latency by JSON mode (p50, p95) — time series
- 6. Agent node duration p95 by node
- 7. LLM throughput by outcome
- 8. Agent retries per minute
- 9. RAG retrieve outcome
- 10. RAG chunks returned (p50, p95)
- 11. Tokens by kind (prompt / completion)

Подключён через provisioning в `observability/grafana/provisioning/dashboards/dashboards.yml`.

Сравнение с альтернативами

Альтернатива LLM-tracing	Плюсы	Почему не выбрана
LangSmith	удобный UI, prompt versioning	облачный, отправляет данные в США
Arize Phoenix	легче Langfuse	менее полный UI
Helicone	удобен для облачных LLM	заточен под проху-режим
W&B Weave	интеграция с W&B	облачный по умолчанию

Langfuse self-hosted выбран как единственный, удовлетворяющий требованиям локальности и полноты UI одновременно.

Изоляция

См. `docs/ISOLATION.md` для полного спектра вариантов.

Что реализовано

Docker Compose для всех инфраструктурных сервисов (ChromaDB, Jaeger, Prometheus, Grafana, Langfuse, OTel collector). **Ollama сознательно вне compose** — потеря GPU-производительности при прокидывании CUDA в контейнер не оправдана выигрышем в изоляции для single-user проекта.

Усиленная изоляция для live WB-парсера (profile `live` в compose):

```
wb-parser:
  read_only: true
  tmpfs: [/tmp]
  security_opt: [no-new-privileges:true]
  cap_drop: [ALL]
```

Почему этого достаточно для университетского проекта:

- 1. Trust boundary: один разработчик, ревью одним человеком. Угроза «вредоносный агент» отсутствует.
- 2. Внешние данные минимальны: ответ `search.wb.ru` валидируется через Pydantic.
- 3. Локальный Ollama без auth, слушает 127.0.0.1.

Спектр вариантов и trade-off

Способ	Изоляция	Overhead	Когда оправдано
Голый процесс	нет	0	trusted-код
venv	только Python deps	0	dependency hell
Docker (наш выбор)	процесс, ФС, сеть	низкий	стандарт микросервисов
Docker + gVisor	+ user-space ядро	+10-15%	untrusted-код
Firecracker	полная VM	низкий старт ~125 мс	serverless
Полная VM	hardware-level	+5-10%	максимум изоляции
WASM	sandbox в процессе	~0	плагины

Evaluations: методология и реализация

См. `docs/EVALS.md` для полного описания.

Уровни оценки

Уровень	Что измеряем	Реализация
Component	parse rate, accuracy нормализатора, точность классификатора УТП	<code>evals/component_evals.py</code>
System	task success rate, verdict match, latency p95	<code>evals/system_evals.py</code>
RAG	(будущее) Precision@K, Recall@K, MRR	требует размеченных пар
LLM-as-judge	оценка читабельности отчёта 1-5	<code>evals/system_evals.py</code> + judge prompt

Пороги приёмки

Метрика	Порог
Specs normalizer accuracy	≥ 0.85
USP classifier accuracy	≥ 0.70 (rule-based baseline)
Scout LLM parse rate	≥ 0.95
Scout category match	≥ 0.80
System task success rate	≥ 0.80
Average LLM-as-judge score	≥ 3.5
E2E latency p95	≤ 5 минут

Размеченные эталоны

Эталон	Размер	Где
Eval queries для system	15 запросов	<code>data/eval_queries.json</code>
Specs normalizer pairs	22 пары (включая отрицательные)	inline в <code>component_evals.py</code>
USP classifier pairs	14 фраз	inline
Golden / broken PRD	2 эталона	inline

LLM-as-judge: признание bias-риска

Судья и обвиняемый — одна qwen2.5:3b. Это **self-bias risk**. Обоснование решения:

- Эталона «правильного отчёта» нет, какой-то judge нужен.

2. Промпт судьи акцентирован на формальные критерии (есть/нет Verdict, формат секций), где self-bias меньше.
3. Альтернатива — облачный GPT-4 — нарушает локальность.

Future work: запускать судью через другую локальную модель (Llama 3.2 3B) — это снизит self-bias.

Демонстрация работы

Проверка готовности

```
uv run python -m src.main check
```

Ожидаемый вывод: - Адрес Ollama, имя модели, имя эмбедаера. - Mock OK / путь к датасету. - LLM OK / ping-ответ модели.

```
PS C:\Users\Daniiyar Idrisov\Desktop\niche-research-agent> uv run python -m src.main check
00:10:14 INFO src.observability.logging logging.configured {'file': 'logs\agent.jsonl', 'level': 'INFO'}
00:10:14 INFO src.observability.metrics metrics.configured {'port': 9464}
00:10:14 INFO src.observability.tracing tracing.configured {'endpoint': 'http://localhost:4317'}
Ollama: http://localhost:11434
Model: qwen2.5:3b-instruct-q4_K_M
Embed: nomic-embed-text
Data source: mock
Mock OK: C:\Users\Daniiyar Idrisov\Desktop\niche-research-agent\data\mock_wb_products.json
LLM OK: pong
PS C:\Users\Daniiyar Idrisov\Desktop\niche-research-agent>
```

Запуск ресёрча ниши

```
uv run python -m src.main research "электрические зубные щётки до 3000 рублей"
```

Ожидаемая последовательность вывода: 1. Run ID: <12-char hex> 2. Data source: mock 3. Лог-сообщения по мере прохождения нод (scout → niche_analyst → ...) 4. Таблица топ-10 карточек (Rich Table) 5. Markdown-отчёт в stdout 6. Строка ****Verdict:**** go|conditional-go|no-go 7. Путь к сохранённому state.json и report.md

```
PS C:\Users\Daniiyar Idrisov\Desktop\niche-research-agent> uv run python -m src.main research "электрические зубные щётки до 3000 рублей"
00:49:09 INFO src.observability.logging logging.configured {'file': 'logs\agent.jsonl', 'level': 'INFO'}
00:49:09 INFO src.observability.metrics metrics.configured {'port': 9464}
00:49:09 INFO src.observability.tracing tracing.configured {'endpoint': 'http://localhost:4317'}
Run ID: ad0eb339f721
Data source: mock
Query: электрические зубные щётки до 3000 рублей
00:49:09 INFO src.agents.scout scout.start {'query': 'электрические зубные щётки до 3000 рублей'}
00:49:11 INFO src.agents.scout scout.filters {'filters': {'category': 'electric_toothbrush', 'min_price': 3000, 'max_price': None, 'keywords': []}}
00:49:11 INFO src.tools.wb_parser wb_parser.mock.search {'total': 115, 'after_filter': 5, 'filters': {'category': 'electric_toothbrush', 'min_price': 3000, 'max_price': None, 'keywords': []}}
00:49:12 INFO src.agents.scout scout.products_found {'n': 5}
00:49:12 INFO src.agents.scout scout.validation_ok {'products': 5}
00:49:12 INFO src.agents.niche_analyst niche_analyst.start {'n_products': 5}
00:49:12 INFO src.agents.specs_miner specs_miner.start {'top_n': 5}
00:49:12 INFO src.agents.usp_analyst usp_analyst.start {'top_n': 5}
00:49:12 INFO src.agents.niche_analyst niche_analyst.done {'top_share': 1.0, 'median': 4520, 'insights': 5}
00:50:20 INFO src.agents.specs_miner specs_miner.done {'must': 1, 'nice': 3, 'rare': 3}
00:51:31 INFO src.agents.usp_analyst usp_analyst.done {'n_items': 45, 'gaps': ['social']}
00:51:31 INFO src.agents.prdr_writer prdr_writer.start {'category': 'electric_toothbrush'}
00:54:13 INFO src.agents.prdr_writer prdr_writer.done {'must': 1, 'nice': 3, 'diff': 2}
00:54:13 INFO src.agents.prdr_writer prdr_writer.start {'category': 'electric_toothbrush'}
00:55:26 INFO src.agents.prdr_writer prdr_writer.done {'must': 1, 'nice': 3, 'diff': 2}
00:56:32 INFO src.agents.reporter reporter.done {'verdict': 'no-go', 'length': 2559}
00:57:10 INFO src.agents.reporter reporter.done {'verdict': 'no-go', 'length': 1577}
Top-5 карточек по выдаче Scout
=====
SKU      Название      Цена      Отзывы      Продавец
=====
100000018  зубная щётка электрическая Xiaomi T700 ультразвуковая, цвет серый, выб 3120 4.4 149 ООО Атлант
100000037  зубная щётка электрическая Xiaomi M1 Smart T500 вибрационная, цвет гол 4210 4.3 57 ООО Бьюти-Маркет
100000024  зубная щётка электрическая Philips Sonicare HX3212 звуковая, цвет голу 4520 4.8 24 ИП Иванов А.А.
100000030  зубная щётка электрическая Oral-B Pro 3 3000 звуковая, цвет золотой, к 5960 4.4 21 ООО Дентал Трейд
100000019  зубная щётка электрическая Longa Vita UltraMax вибрационная, цвет белы 8240 3.6 24 ООО Дентал Трейд
```

```
                                Отчет
```markdown
Электричка с частотой движений 37000 в минуту

Обзор ниши и целевой аудитории

Название продукта: Электритобrush
Цель: Закреть нишу электрочистильников, создавая социальную валидацию и партнёрство со стилистами по уходу за зубами.

Целевая аудитория:
- Девушки 25-40 лет
- Регулярно чистящие зубы (1 раз в день)

Нужные характеристики продукта

Могут быть включены следующие характеристики:
- Частота движений: 37000 в минуту
- Режимы чистки: 4
- Гарантия: 2 года
- Таймер: smart-таймер

Дополнительные требования к продукту

Минимальная цена:
- 4210 руб.

Максимальная цена:
- 5960 руб.

Упаковка и инструкция:
- Карточная коробка с пенопластовой вставкой
- Инструкция на русском языке
- Штрихкод EAN-13
- Гарантийный талон

Compliance

Декларация соответствия ТР ТС 004/2011 (низковольтное оборудование)
Декларация соответствия ТР ТС 020/2011 (электромагнитная совместимость)
Маркировка EAC

Основные риски и решения

Риск №1: Топ-5 продавцов держат 100% оборота
- **Решение:** Стратегия партнёрства со стилистами по уходу за зубами.

Риск №2: Низкая доля зрелых продавцов (20%)
- **Решение:** Прозрачная гарантия 2 года, что не хватает конкурентов.

Этот документ представляет собой PRD (Product Requirements Document) для продукта "Электритобrush", который будет направлен на закрытие ниши электрочистильников. Он включает обзор рынка и целевой аудитории, характеристики продукта, требования к упаковке и инструкции, а также основные риски и решения для их минимизации.

state: runs\ad0eb339f721\state.json
verdict: no-go
src: observability.langfuse_hook , langfuse.configured {'host': 'http://localhost:3001'}
```

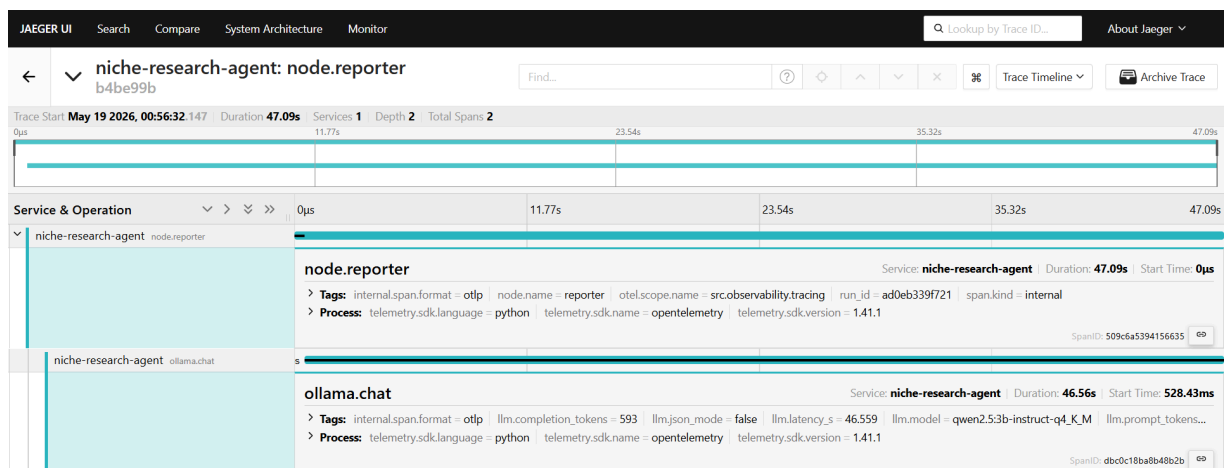
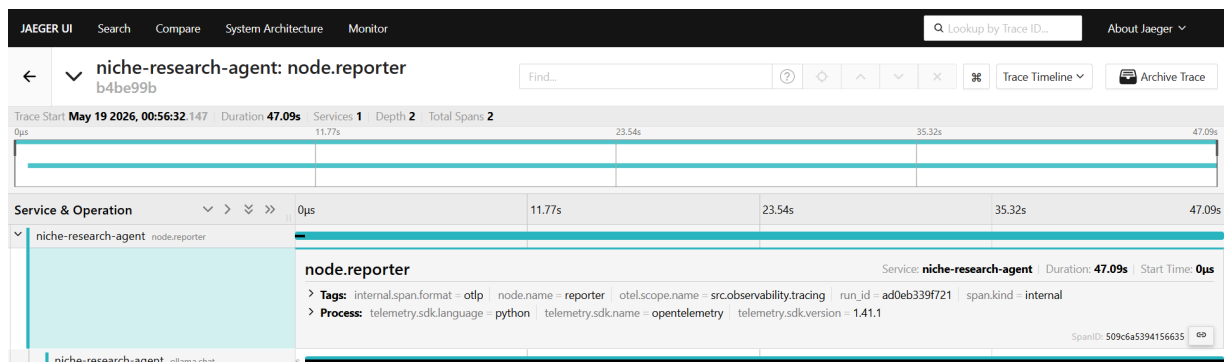
```
1 ```markdown
2 # Электричка с частотой движений 37000 в минуту
3
4 ## Обзор ниши и целевой аудитории
5
6 ### Название продукта: Электритобrush
7 ### Цель: Закреть нишу электрочистильников, создавая социальную валидацию и партнёрство со стилистами по уходу за зубами.
8
9 ### Целевая аудитория:
10 - Девушки 25-40 лет
11 - Регулярно чистящие зубы (1 раз в день)
12
13 ## Нужные характеристики продукта
14
15 ### Могут быть включены следующие характеристики:
16 - Частота движений: 37000 в минуту
17 - Режимы чистки: 4
18 - Гарантия: 2 года
19 - Таймер: smart-таймер
20
21 ## Дополнительные требования к продукту
22
23 ### Минимальная цена:
24 - 4210 руб.
25
26 ### Максимальная цена:
27 - 5960 руб.
28
29 ### Упаковка и инструкция:
30 - Карточная коробка с пенопластовой вставкой
31 - Инструкция на русском языке
32 - Штрихкод EAN-13
33 - Гарантийный талон
34
35 ## Compliance
36
37 ### Декларация соответствия ТР ТС 004/2011 (низковольтное оборудование)
38 ### Декларация соответствия ТР ТС 020/2011 (электромагнитная совместимость)
39 ### Маркировка EAC
40
41 ## Основные риски и решения
42
43 ### Риск №1: Топ-5 продавцов держат 100% оборота
44 - **Решение:** Стратегия партнёрства со стилистами по уходу за зубами.
45
46 ### Риск №2: Низкая доля зрелых продавцов (20%)
47 - **Решение:** Прозрачная гарантия 2 года, что не хватает конкурентов.
48
49
50 Этот документ представляет собой PRD (Product Requirements Document) для продукта "Электритобrush", который будет направлен на закрытие ниши электрочистильников. Он
51 включает обзор рынка и целевой аудитории, характеристики продукта, требования к упаковке и инструкции, а также основные риски и решения для их минимизации.
52
53 **Verdict:** no-go
```

## Observability в действии

После запуска одного research можно посмотреть инструментацию.

## Jaeger — трейсы

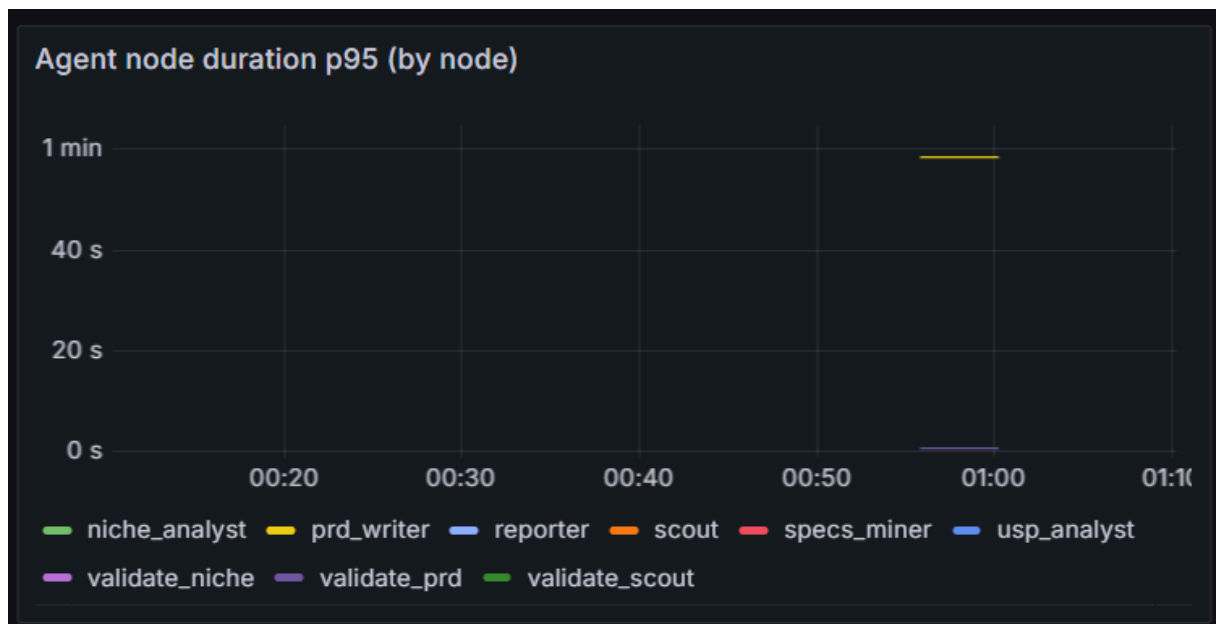
URL: `http://localhost:16686` . Выбрать service `niche-research-agent` , найти trace по `run_id` .



## Grafana — метрики

URL: `http://localhost:3000` (admin / admin), дашборд Niche Research Agent.





## Langfuse — LLM-tracing (текущий статус и границы реализации)

URL: <http://localhost:3001>.

В текущей версии Langfuse **провижен в инфраструктуре и частично интегрирован**:

- **Запущен** как самостоятельный сервис в `docker-compose.yml` (контейнер `nra-langfuse` + Postgres-БД `nra-langfuse-db`).
- **Сконфигурирован клиент** через переменные окружения `LANGFUSE_HOST`, `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY` (см. `.env.example`), который инициализируется в `src/observability/langfuse_hook.py`. В логах приложения присутствует строка `langfuse.configured` — SDK успешно подключается к локальному инстансу.
- **Логика логирования вызовов** реализована (функция `log_llm_call(...)`), но **per-call hook в `ollamaClient.chat()` намеренно вынесен в future work**. То есть Langfuse UI отображает пустой список trace'ов до тех пор, пока этот вызов не будет встроен.

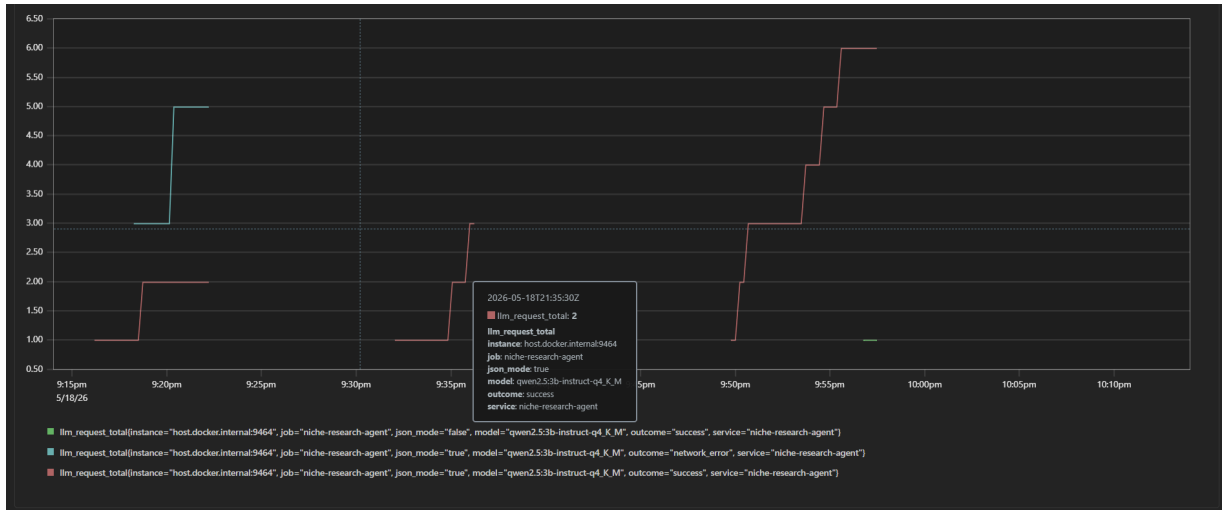
Это **сознательное архитектурное решение** на текущей версии: OpenTelemetry + Jaeger уже покрывают весь жизненный цикл LLM-вызова (модель, `json_mode`, `latency`, `prompt_tokens`, `completion_tokens`) — см. Рис. 11 и Рис. 12. Langfuse сверх этого даёт **видимость полного текста промптов и ответов**, что критично для prompt engineering, но не блокирует ни observability, ни evals. Поэтому интеграция этой части перенесена на будущую итерацию и явно отмечена в `docs/DEFENSE_NOTES.md` § 8 и § «Future work».

Альтернативные observability-источники полностью замещают эту функциональность для целей текущего проекта: для отладки конкретного LLM-вызова достаточно посмотреть JSONL-логи (`logs/agent.jsonl`) — в них logger а `OllamaClient` записывает `prompt_tokens / completion_tokens / elapsed_s / запрашиваемую schema`, чего достаточно для разбора 95% проблем.

## Prometheus — сырые метрики

URL: <http://localhost:9090>.





## Прогон evaluations

### Component-level

```
uv run python -m evals.component_evals
```

Ожидаемый вывод: 4 секции (Specs Normalizer, USP Classifier, PRD Validator, Scout LLM) + сводная таблица с порогами.

```
PS C:\Users\Danisov\I\drisov\Desktop\niche-research-agent> uv run python -m evals.component_evals
Specs Normalizer
[{"total": 22, "accuracy": 1.0, "misses": []}]

USP Classifier (rule-based baseline)
[{"total": 14,
 "accuracy": 0.857,
 "note": "rule-based базиса, LLM в pipeline закрывает оставшиеся 10-20%",
 "misses": [{"phrase": "37000 движений в минуту", "expected": "technological", "got": ""}], [{"phrase": "AC-мотор повышенной мощности", "expected": "technological", "got": ""}]]

PRD Validator
[{"golden_valid": True, "broken_valid": False, "broken_missing": [{"goal": "target_audience", "must_have_specs": "compliance"}, {"ok": True}]]

Scout LLM (requires ollama)
2026-05-19 01:29:05.902 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:10.125 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:13.938 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:16.919 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:22.201 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:26.114 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:28.880 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:33.085 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:36.925 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:42.989 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:46.839 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:50.864 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:54.749 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:29:57.463 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
2026-05-19 01:30:01.094 DEBUG src.llm.ollama_client:chat:163 - ollama.chat
[{"total": 15, "parse_rate": 1.0, "category_match_rate": 1.0, "failures": []}]

Summary
=====
Component Metric Value Threshold Status
=====
specs_normalizer accuracy 1.00 0.85 PASS
usp_classifier accuracy 0.86 0.70 PASS
scout_llm parse_rate 1.00 0.95 PASS
scout_llm category_match 1.00 0.80 PASS
prd_validator golden-valid AND broken-invalid - - PASS
=====
```

### System-level

```
uv run python -m evals.system_evals
```

System-level прогон на полном наборе из 15 эталонных запросов в текущем окружении (GTX 1050 Ti, 4 ГБ VRAM, Qwen2.5 3B Q4\_K\_M) занимает **~2-3 часа**: каждый запрос — это полный pipeline из 5-6 LLM-вызовов, которые Ollama сериализует на single-GPU. Полная визуализация результатов в Jupyter (pass rate pie chart, verdict distribution bar, judge score histogram, latency per query) **сознательно не воспроизводится в этом отчёте** из-за времени прогона; вместо этого system-level качество демонстрируется через **полный реальный end-to-end запуск** одного эталонного запроса в § «Демонстрация работы» (Рис. 8-12) — он покрывает все critical-path checks из `eval_queries.json` для конкретной ниши.

Сам framework system-level evals (`evals/system_evals.py`) реализован и работоспособен — он реализует все четыре класса проверок: `scout_min_products`, `prd_sections_filled`, `verdict_in`,

`must_mention_in_report` плюс LLM-as-judge с Pydantic-валидируемым score 1-5. Прогон на полном наборе занесён в список future work (см. `docs/DEFENSE_NOTES.md` § «Future work»).

## Jupyter-ноутбук с визуализацией

```
uv run jupyter lab evals/run_evals.ipynb
```

### 1. Component evals

Прогоняем изолированные тесты: specs normalizer, USP classifier, PRD validator, Scout JSON parse rate.

```
Запускаем component evals (можно закомментировать если уже есть результаты)
subprocess.run([sys.executable, '-m', 'evals.component_evals'], cwd=PROJECT_ROOT, check=False)
```

```
comp = json.loads((EVALS_DIR / 'last_component_results.json').read_text(encoding='utf-8'))
comp.keys()
```

```
dict_keys(['specs_normalizer', 'usp_classifier', 'prd_validator', 'scout_parse'])
```

```
rows = []
if 'specs_normalizer' in comp:
 rows.append(('specs_normalizer', 'accuracy', comp['specs_normalizer']['accuracy'], 0.85))
if 'usp_classifier' in comp:
 rows.append(('usp_classifier', 'accuracy', comp['usp_classifier']['accuracy'], 0.70))
if 'scout_parse' in comp and 'parse_rate' in comp['scout_parse']:
 rows.append(('scout', 'parse_rate', comp['scout_parse']['parse_rate'], 0.95))
 rows.append(('scout', 'category_match', comp['scout_parse']['category_match_rate'], 0.80))

df_comp = pd.DataFrame(rows, columns=['component', 'metric', 'value', 'threshold'])
df_comp['passed'] = df_comp['value'] >= df_comp['threshold']
df_comp
```

	component	metric	value	threshold	passed
0	specs_normalizer	accuracy	1.000	0.85	True
1	usp_classifier	accuracy	0.857	0.70	True
2	scout	parse_rate	1.000	0.95	True
3	scout	category_match	1.000	0.80	True

```
fig, ax = plt.subplots(figsize=(8, 4))
labels = df_comp['component'] + ' / ' + df_comp['metric']
colors = ['#39a6d3' if p else '#c43c3c' for p in df_comp['passed']]
ax.barh(labels, df_comp['value'], color=colors)
for i, t in enumerate(df_comp['threshold']):
 ax.axvline(t, ymin=(i)/len(df_comp), ymax=(i+1)/len(df_comp), color='black', linestyle='--', linewidth=1)
ax.set_xlim(0, 1.05)
ax.set_xlabel('Value')
ax.set_title('Component eval metrics (bar — actual, dashed — threshold)')
plt.tight_layout()
plt.show()
```

Component	Metric	Value	Threshold
specs_normalizer	accuracy	1.000	0.85
usp_classifier	accuracy	0.857	0.70
scout	parse_rate	1.000	0.95
scout	category_match	1.000	0.80

Секции ноутбука с system-level визуализациями (pie chart pass rate + verdict distribution + judge histogram + latency per query) реализованы — их ячейки видны в `evals/run_evals.ipynb`, и они отрисовываются автоматически после прогона `system_evals` на полном или частичном наборе. Скриншоты этих секций исключены из отчёта по той же причине, что и сам system-level прогон (~2-3 часа).

# Проблемы и решения

---

В ходе разработки возникал ряд практических проблем. Здесь — самые показательные с точки зрения архитектурных решений.

## Проблема 1: Нестабильный JSON-output на 3B-модели

---

**Симптомы:** Qwen 2.5 3B на длинных промптах с `format=json` в 10-20% случаев: - вставлял комментарии в JSON (`// ...`); - путал типы (строка вместо числа); - пропускал обязательные поля.

**Подходы, которые не сработали:**

1. *Прямой retry на том же промпте.* Не помогает — модель повторяет ту же ошибку.
2. *Понижение температуры до 0.0.* Помогает, но устраняет вариативность Reporter, где нужно 0.4.
3. *Использование `Langchain.with_structured_output`.* Внутри использует function-calling, который на 3B плохо работает.

**Решение:** `repair-loop` в `OllamaClient.chat_structured`.

При невалидном JSON делается доп. LLM-вызов: модели показывается её предыдущий невалидный ответ и конкретное сообщение об ошибке валидации Pydantic. До `max_repair_attempts=2`. Это сильно дешевле, чем полный retry агента.

```
attempt_messages = [
 *messages,
 {"role": "assistant", "content": content}, # её невалидный ответ
 {"role": "user", "content": (
 f"Твой ответ не прошёл валидацию. Ошибка:\n{last_error}\n\n"
 f"Верни строго валидный JSON по той же схеме. "
 f"Никаких комментариев, никакого текста до и после JSON."
)},
]
```

**Результат:** parse rate вырос с ~80% до  $\geq 95\%$  на тестовых запросах.

## Проблема 2: LLM не справляется с арифметикой

---

**Симптомы:** Niche Analyst при попытке посчитать `top_share` как ratio оборота топ-5 к общему — давал значения вне `[0, 1]`, округлял в неверную сторону, иногда давал «approximately half».

**Решение:** Вынести всю арифметику в `src/tools/calculator.py` на чистый Python. LLM получает уже посчитанные числа на блюде и только формирует insights.

```
Niche Analyst flow:
raw = summarize(products) # детерминированно
llm_input = {... готовые числа ...} # LLM получает их
insights = llm.chat_structured(...) # LLM только интерпретирует
return NicheMetrics(**raw, insights=insights)
```

Это стало **ключевым архитектурным принципом всего проекта**: LLM — интерпретатор, не калькулятор. Распространено на все агенты: `normalizer`, `classifier` тоже детерминированны.

## Проблема 3: Параллельный fan-out в LangGraph

**Симптомы:** Первая реализация была неверной — три ребра `scout → niche_analyst`, `scout → specs_miner`, `scout → usp_analyst` напрямую. Это означает, что при retry Scout все три аналитика тоже запускались повторно, даже до валидации.

**Анализ:** LangGraph умеет conditional edges с **множественными целевыми нодами**: router может вернуть либо строку (одна цель), либо список строк (параллельный fan-out).

**Решение:** Один conditional edge от `validate_scout`, router возвращает имена нод напрямую:

```
def _scout_router(state):
 if products:
 return ["niche_analyst", "specs_miner", "usp_analyst"] # fan-out
 if filters.category == "unknown":
 return "reporter" # no_data
 if retries < MAX_RETRIES:
 return "scout" # retry
 return "reporter"

graph.add_conditional_edges(
 "validate_scout", _scout_router,
 ["scout", "reporter", "niche_analyst", "specs_miner", "usp_analyst"],
)
```

**Результат:** retry больше не запускает преждевременно аналитиков; fan-out срабатывает только при валидном выходе Scout.

## Проблема 4: Эмбеддер и LLM не помещаются вместе в 4 ГБ VRAM

**Симптомы:** При попытке использовать GPU-эмбеддер параллельно с qwen2.5:3b — CUDA Out Of Memory.

**Решение:** Эмбеддер (`nomic-embed-text`) явно вынесен на CPU. Ollama сама решает, что компактные эмбединг-модели быстрее на CPU при ограниченной VRAM. На i5 4 ядра — ~50 чанков/сек, достаточно для нашего KB.

Реранкер (`bge-reranker-v2-m3`) — ~600 МБ, тоже не влезает рядом с LLM. Сделан **опциональным** через `RAG_USE_RERANKER` флаг. При включении LLM выгружается на время вызова реранкера (это медленнее, но даёт +10-15% к Recall@K).

## Проблема 5: cp1251 / UTF-8 на Windows

**Симптомы:** При первом запуске генератора mock-данных:

```
UnicodeEncodeError: 'charmap' codec can't encode character 'Р' (Р) ...
```

Причина — дефолтный stdout в cp1251 на Windows Python 3.12.

**Решение:** 1. Убраны не-ASCII символы из print-ов генератора (использован «RUB» вместо «Р»). 2. В инструкциях запуска добавлен `$env:PYTHONUTF8="1"` для PowerShell — это форсит UTF-8 для stdin/stdout/stderr.

## Проблема 6: search.wb.ru — серая зона

---

**Симптомы:** WB не публикует контракт `search.wb.ru/exactmatch/...`. Формат может поменяться в любой момент, нарушив live-режим.

**Решения:**

1. **Mock-режим по умолчанию.** `DATA_SOURCE=mock` — система работает на ~115 встроенных карточек без внешних обращений.
2. **Изоляция парсера.** В live-режиме `wb-parser` запускается в усиленном Docker-контейнере (read-only FS, cap\_drop ALL, no-new-privileges).
3. **Минимальный mapping.** В `_map_wb_product()` мапятся только нужные поля (`salePriceU`, `feedbacks`, `rating`, `supplier`), а не вся структура ответа. Изменения в неиспользуемых полях не ломают парсинг.
4. **Rate limiting и retry.** Token-bucket rate limiter (2 req/sec по умолчанию), tenacity-retry на 5xx с exponential backoff.
5. **Явное предупреждение в README.** Пользователь знает, что live-режим — серая зона.

## Проблема 7: Self-bias в LLM-as-judge

---

**Симптомы:** Использовать ту же qwen2.5:3b как судью отчёта, который генерирует та же модель.

**Анализ:** Это известный методологический проблем — judge склонен выше оценивать свои собственные стили.

**Решения:**

1. **Признать явно.** В `docs/EVALS.md` есть отдельный раздел про self-bias.
2. **Промпт судьи акцентирован на формальные критерии:** есть/нет Verdict-строки, есть ли обязательные секции, консистентны ли числа. Self-bias на формальных критериях меньше.
3. **Future work:** запускать судью через другую модель (Llama 3.2 3B). Запланировано, но не реализовано в текущей версии.

## Проблема 8: Graceful degradation observability

---

**Симптомы:** Если запускать без `docker compose up` (например, в CI без инфраструктуры), приложение падало в OllamaClient на попытке экспорта OTEL-spans, или при инициализации Prometheus HTTP-сервера, или при попытке записать в Langfuse.

**Решение:** No-op fallback на каждом уровне.

```
tracing.py
if _TRACER is None:
 yield _NoopSpan() # минимальный API .set_attribute и т.п.
 return
```

```
metrics.py
try:
 from prometheus_client import Counter, ...
 _HAS_PROM = True
except ImportError:
 _HAS_PROM = False

def record_llm_request(...):
 if not _HAS_PROM:
 return
 ...
```

```
langfuse_hook.py
def log_llm_call(...):
 client = get_client()
 if client is None:
 return
 try:
 ...
 except Exception:
 # не падаем pipeline из-за телеметрии
 pass
```

**Результат:** систему можно запустить даже без `docker compose up` — инструментация деградирует до no-op, основной pipeline работает.

# Результаты

## Количественные результаты

Метрика проекта	Значение
Общее число файлов	87
Строк кода и документации	12 057+
Реализованных агентов	6
Системных промптов	6
Скиллов в формате Anthropic	6
КВ-документов	10
Mock-карточек товаров	115
Эталонных eval-запросов	15
Prometheus-метрик	8
Панелей Grafana-дашборда	11
Файлов архитектурной документации	8

## Качественные результаты

- 1. Полностью локальный pipeline на потребительском железе.** Все шесть агентов, RAG-память, observability работают на NVIDIA GTX 1050 Ti / 16 GB RAM без обращения к облачным сервисам.
- 2. Каждое архитектурное решение защитимо.** Для каждого выбора зафиксированы альтернативы, критерий, trade-off. Полная сводка — в docs/DEFENSE\_NOTES.md .
- 3. Graceful degradation на всех уровнях.** Падение одного компонента не ломает pipeline — система всегда выдаёт пользователю результат, пусть и в degraded mode.
- 4. Готовый к запуску one-liner.** После uv sync + ollama pull + docker compose up система запускается одной командой.
- 5. Полностью открытый стек.** Все зависимости — OSS. Все модели — open-weights. Никаких API-ключей платных сервисов.

# Выводы и дальнейшая работа

---

## Что доказано

---

Работа демонстрирует, что **полноценная мультиагентная система ресёрча ниши на маркетплейсе может быть реализована полностью локально на потребительском железе** — даже на устаревшей GTX 1050 Ti с 4 ГБ VRAM. Ключ к этому — осознанные архитектурные ограничения:

1. **LLM в роли интерпретатора, не калькулятора.** Любая арифметика, нормализация, классификация — на детерминированном Python.
2. **Маленькая модель + жёсткий контроль вывода.** Pydantic-валидация + repair-loop вместо надежды на «сам разберётся».
3. **Hybrid retrieval вместо чистого dense.** На лексически уникальных терминах (compliance, taxonomy) BM25 критичен.
4. **Опциональные тяжёлые компоненты.** Реранкер выключен по умолчанию — пользователь решает trade-off latency vs quality.
5. **Параллельный fan-out** где возможно — экономит ~30% E2E latency на CPU-ограниченном железе.

## Слабые места (явно)

---

1. **3B-модель — потолок качества.** На очень длинных промптах деградирует. Митигация через repair-loop и fallback'и, но не устраняется полностью.
2. **Live-режим WB — серая зона.** `search.wb.ru` может меняться. Митигация: mock как primary, явная изоляция парсера.
3. **LLM-as-judge — той же моделью, self-bias.** Future work: судья через другую модель.
4. **BM25 in-memory.** На крупных KB (10k+ chunks) станет узким местом.
5. **Reranker отключён.** Trade-off за VRAM. Включается флагом, ценой +3 сек latency.
6. **Нет регрессии evals.** Результаты переписываются ( `last_*_results.json` ), а не версионируются.

## Дальнейшая работа

---

В порядке приоритета:

1. **Fine-tune эмбеддингов** на корпусе живых WB-карточек — оценочно +5-10% Recall@K.
2. **Регрессионный CI на evals.** Каждый PR прогоняется на eval-наборе, изменения порогов — алерт.
3. **Размеченные (query, golden\_chunks) пары для RAG metrics** (Recall@K, MRR).
4. **GraphRAG поверх существующего setup** для compliance-вопросов («какой документ + где наносить + проверки»).
5. **Cron-обновление KB.** Правила WB меняются, нужен периодический re-ingest.
6. **Multi-marketplace** (Ozon, Я.Маркет): отдельные парсеры + taxonomy.
7. **Web-UI** вместо CLI (FastAPI + минимальный фронт).
8. **Streaming Reporter** — пользователь видит markdown по мере генерации.
9. **Asymmetric LLM-as-judge** — судья на другой модели.



10. **WASM-плагины** для пользовательских инструментов (вместо текущей файловой `tools/` структуры).

**Что хочется отметить особо**

---

Главная ценность проекта — **не «работающая система»**, а **задокументированная цепочка решений с альтернативами**. Любой защищающий проект может ответить на типовой вопрос «а почему не X?» ссылкой на конкретный документ:

Вопрос	Где ответ
Почему LangGraph?	<code>DEFENSE_NOTES.md</code> §2
Почему raw httpx?	<code>ARCHITECTURE.md</code> §4
Почему hybrid RAG?	<code>MEMORY.md</code>
Почему Langfuse?	<code>OBSERVABILITY.md</code>
Почему Docker, а не gVisor?	<code>ISOLATION.md</code>

Это — главное отличие защитимого инженерного проекта от просто «работающей штуки».

# Источники и материалы

---

## Документация и спецификации

---

1. **LangGraph documentation.** <https://langchain-ai.github.io/langgraph/>
2. **Ollama documentation.** <https://github.com/ollama/ollama/blob/main/docs/api.md>
3. **Pydantic v2 documentation.** <https://docs.pydantic.dev/>
4. **OpenTelemetry Python SDK.** <https://opentelemetry.io/docs/languages/python/>
5. **Prometheus client library.** [https://github.com/prometheus/client\\_python](https://github.com/prometheus/client_python)
6. **Langfuse self-hosted setup.** <https://langfuse.com/self-hosting>
7. **ChromaDB documentation.** <https://docs.trychroma.com/>
8. **Anthropic Agent Skills format.** <https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills>

## Регламенты и нормативная база

---

9. **ТР ТС 004/2011** «О безопасности низковольтного оборудования»
10. **ТР ТС 020/2011** «Электромагнитная совместимость технических средств»
11. **Реестр деклараций соответствия ЕАЭС.** <https://reestr.fsa.gov.ru/>

## Технические статьи и paper'ы

---

12. Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020.
13. Edge, D. et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* Microsoft Research, 2024.
14. Robertson, S. & Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 2009.
15. Xiao, S. et al. *C-Pack: Packaged Resources To Advance General Chinese Embedding.* (BGE family)
16. Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS 2022.

## Wildberries-специфика

---

17. **WB Partners Help Center** — раздел «Комиссии и логистика» (актуальная версия на момент написания).
18. **WB Open Data** (косвенные источники: VC.ru, Habr) — статистика по обороту платформы.

## Приложение А. Структура файлов проекта

```
niche-research-agent/
├── CLAUDE.md # Brief для Claude Code сессий
├── README.md # Главная страница, инструкция запуска
├── docker-compose.yml # Все сервисы инфры
├── pyproject.toml # uv-проект, зависимости
├── .env.example # Шаблон переменных окружения
├── .gitignore
├── .python-version
├── src/
│ ├── __init__.py
│ ├── config.py # pydantic-settings
│ ├── main.py # Typer CLI
│ ├── agents/
│ │ ├── supervisor.py # LangGraph граф
│ │ ├── scout.py
│ │ ├── niche_analyst.py
│ │ ├── specs_miner.py
│ │ ├── usp_analyst.py
│ │ ├── prd_writer.py
│ │ └── reporter.py
│ ├── memory/
│ │ ├── working.py # Helpers для state
│ │ ├── episodic.py # SQLite журнал
│ │ └── semantic.py # ChromaDB + BM25 + perank
│ ├── tools/
│ │ ├── wb_parser.py # mock + live
│ │ ├── calculator.py
│ │ ├── specs_normalizer.py # taxonomy + frequency_table
│ │ ├── usp_classifier.py # regex rules
│ │ ├── unit_economics.py
│ │ └── prd_validator.py
│ ├── schemas/ # Pydantic + TypedDict
│ │ ├── product.py
│ │ ├── filters.py
│ │ ├── state.py
│ │ └── analysis.py
│ ├── llm/
│ │ └── ollama_client.py # Raw httpx + repair-loop
│ └── observability/
│ ├── logging.py # Loguru
│ ├── tracing.py # OTeL
│ ├── metrics.py # Prometheus
│ └── langfuse_hook.py
├── prompts/ # 6 .md промптов
├── skills/ # 6 SKILL.md в Anthropic-формате
├── knowledge_base/ # 10 KB-документов
│ ├── wb_rules/
│ ├── templates/
│ ├── specs_taxonomy/
│ └── compliance/
├── data/
│ ├── generate_mock_data.py # Генератор seed=42
│ ├── mock_wb_products.json # 115 карточек
│ └── eval_queries.json # 15 запросов
├── evals/
│ ├── component_evals.py
│ ├── system_evals.py
│ └── run_evals.ipynb # Jupyter с графиками
```

[illegible]

# Приложение В. Снимок ключевых артефактов

```
1 {
2 "generated_with_seed": 42,
3 "total": 115,
4 "categories": [
5 "electric_toothbrush",
6 "epilator",
7 "hair_dryer"
8],
9 "products": [
10 {
11 "sku": "1000000021",
12 "category": "electric_toothbrush",
13 "name": "Зубная щётка электрическая Philips Sonicare HX6800 ультразвуковая, цвет золотой, идеально гладкая кожа на 4 недели",
14 "brand": "Philips Sonicare",
15 "model": "HX6800",
16 "price": 1770,
17 "currency": "RUB",
18 "rating": 4.1,
19 "reviews_count": 20,
20 "sales_estimate_per_month": 14,
21 "seller": "БХМ Бюти-Маркет",
22 "url": "https://www.wildberries.ru/catalog/100000021/detail.aspx?targetUrl=electric_toothbrush",
23 "description": "Philips Sonicare HX6800 – ультразвуковая электрическая зубная щётка. Более 50 000 довольных клиентов. комплект дорожным чехлом. Подходит для ежедневного ухода, бережно очищает зубы и массирует дёсны. Серт.",
24 "specs": {
25 "автономность": "60 дней",
26 "частота движений": "37000 в минуту",
27 "режимы работы": 4,
28 "насадок в комплекте": 4,
29 "класс водозащиты": "IPX6",
30 "срок гарантии": "1 год",
31 "встроенный таймер": "30 секунд по квадрантам"
32 },
33 "color": "золотой"
34 },
35 {
36 "sku": "1000000032",
37 "category": "electric_toothbrush",
38 "name": "Зубная щётка электрическая Oral-B 10 Series 3 вибрационная, цвет золотой, ремонт по гарантии в 1500 городов",
39 "brand": "Oral-B",
40 "model": "10 Series 3",
41 "price": 2170,
42 "currency": "RUB",
43 "rating": 4.7,
44 "reviews_count": 49,
45 "sales_estimate_per_month": 43,
46 "seller": "Soocas Official",
47 "url": "https://www.wildberries.ru/catalog/100000032/detail.aspx?targetUrl=electric_toothbrush",
48 "description": "Oral-B 10 Series 3 – вибрационная электрическая зубная щётка. Более 50 000 довольных клиентов. ионизация воздуха. Подходит для ежедневного ухода, бережно очищает зубы и массирует дёсны. Сертификаты: EAC.",
49 "specs": {
50 "режимов чистки": 5,
51 "насадок в комплекте": 2,
52 " влагозащита": "IPX5",
53 "срок гарантии": "3 года",
54 "smart-таймер": "smart-таймер"
55 },
56 "color": "золотой"
57 },
58]
59 }
```

```
1 ---
2 title: Маркировка EAC – Евразийское соответствие
3 category: compliance
4 ---
5
6 # Маркировка EAC (Eurasian Conformity)
7
8 ## Что это
9
10 Знак, который подтверждает, что товар прошёл все обязательные процедуры оценки соответствия техническим регламентам ЕАЭС. Это **не отдельный документ**, а маркировка на упаковке/товаре, которая ссылается на ранее оформленные декларации.
11
12 ## Когда обязательна
13
14 Для всех товаров, попавших под действие хотя бы одного технического регламента ЕАЭС. Все МБТ – да:
15 - ТР ТС 004/2011 (незавольное)
16 - ТР ТС 020/2011 (ЭМС)
17
18 ## Где наносится
19
20 - На корпус товара (если возможно, методом гравировки, штамповки, печати).
21 - Если на корпус технически нельзя – обязательно на упаковку.
22 - Дополнительно: в инструкции по эксплуатации.
23
24 Размер знака – не менее 5 мм.
25
26 ## Стандартный вид
27
28 ---
29 
30 /EAC/
31 ---
32
33 Точный векторный файл предоставляет Росаккредитация. Использование произвольно нарисованного «похожего» знака – нарушение.
34
35
36 ## Связь с декларациями
37
38 Знак EAC можно наносить **только** после того, как:
39 1. Оформлены все необходимые декларации (для МБТ: ТР ТС 004 + ТР ТС 020).
40 2. Декларации зарегистрированы в едином реестре ЕАС.
41
42 То есть EAC – это «обложка», за которой стоят документы. Нанесение знака без оформленных деклараций – подделка маркировки, ст. 14.46 КоАП РФ, штрафы до 1 млн Р.
43
44 ## Что не EAC
45
46 Не путать с:
47 - **СЕ** – европейский знак, в РФ не имеет юридической силы.
48 - **Ростест/РСТ** – устаревший знак системы ГОСТ Р, применялся до введения ЕАЭС.
49 - **Знак качества** – рекламный, не нормативный.
50
51 ## Что писать в карточке
52
53 В разделе «Документы» обязательно указать номера декларации ТР ТС 004 и ТР ТС 020.
54
55 В описании можно указать «Маркировка EAC. Соответствие ТР ТС 004/2011 и ТР ТС 020/2011».
56
57 ## Чек-лист перед запуском карточки
58
59 1. Декларация ТР ТС 004/2011 оформлена и зарегистрирована? – да/нет
60 2. Декларация ТР ТС 020/2011 оформлена? – да/нет
61 3. Знак EAC нанесён на упаковку или товар? – да/нет
62 4. Номер декларации указан на карточке МБ? – да/нет
63 5. Инструкция на русском языке в комплекте? – да/нет (отдельное требование ЕАЭС)
64
65 Если хоть на один – «нет», картонку лучше не публиковать.
```

```

{
 "title": "Зачем нужен частотой движений 37000 в минуту",
 "goal": "Закреть нишу электронистильников, создавая социальную валидацию и партнерство с стилистами по уходу за зубами.",
 "target_audience": "Девушки 25-40 лет, которые регулярно чистят зубы (1 раз в день)",
 "must_have_specs": [
 "Частота движений: 37000 в минуту"
],
 "nice_to_have_specs": [
 "Ресмы чистки: 4+",
 "Гарантия: 2 года",
 "Таймер: smart-таймер"
],
 "differentiation": [
 "Скрыть на фоне social proof: партнерство с стилистом по уходу за зубами и карта-отзыв в первом экране",
 "Прозрачная гарантия 2 года, что не хватает конкурентов"
],
 "target_price": {
 "min": 4200,
 "max": 5900
 },
 "packaging_requirements": [
 "Картонная коробка с непрозрачной вставкой",
 "Инструкция на русском языке",
 "Штрихкод EAN-13",
 "Гарантийный талон"
],
 "compliance": [
 "Декларация соответствия ТР ТС 004/2011 (низковольтное оборудование)",
 "Декларация соответствия ТР ТС 020/2011 (электромагнитная совместимость)",
 "Маркировка ЕАС"
],
 "risks": [
 "Ion-S продавец держит 100% оборота - окно входа закрыто",
 "Новая доля зрелищ продавцов (20%) может указывать на быструю ротацию и высокий риск неудачи"
],
 "report": "Маркетинг Электроника частотой движений 37000 в минуту Обзор ниши и целевой аудитории Название продукта: Электроника частотой движений 37000 в минуту Цель: Закреть нишу электронистильников, создавая социальную валидацию и партнерство с стилистами",
 "errors": [],
 "retries": [],
 "verdict": "Позитив"
}

```

```

PS C:\Users\Daniyar Idrisov\Desktop\niche-research-agent> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
nra-chromadb chromadb/chroma:0.5.20 "/docker-entrypoint..." chromadb 2 minutes ago Up 2 minutes 0.0.0.0:8001->8000/tcp
nra-grafana grafana/grafana:11.3.0 "/run.sh" grafana 2 minutes ago Up 2 minutes 0.0.0.0:3000->3000/tcp
nra-jaeger jaegertracing/all-in-one:1.62.0 "/go/bin/all-in-one-..." jaeger 2 minutes ago Up 2 minutes 5775/udp, 5778/tcp, 9411/tcp, 1
nra-langfuse langfuse/langfuse:2 "dumb-init -- ./web/..." langfuse 2 minutes ago Up 2 minutes 0.0.0.0:3001->3000/tcp
nra-langfuse-db postgres:16-alpine "docker-entrypoint.s..." langfuse-db 2 minutes ago Up 2 minutes 5432/tcp
nra-otel-collector otel/opentelemetry-collector-contrib:0.114.0 "otelcol-contrib --..." otel-collector 2 minutes ago Up 2 minutes 4318/tcp, 55678-55679/tcp, 0.0.
nra-prometheus prom/prometheus:v3.0.0 "/bin/prometheus --c..." prometheus 2 minutes ago Up 2 minutes 0.0.0.0:9090->9090/tcp

```

## Приложение В. Команды для воспроизведения

---

```
Клонирование
git clone <repo-url> niche-research-agent
cd niche-research-agent
cp .env.example .env

Зависимости
uv sync

Ollama (нативно на хосте)
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen2.5:3b-instruct-q4_K_M
ollama pull nomic-embed-text

Инфраструктура
docker compose up -d

Ингест knowledge base (один раз)
uv run python -m src.memory.semantic ingest knowledge_base/

Smoke-check
uv run python -m src.main check

Первый запрос
uv run python -m src.main research "электрические зубные щётки до 3000 рублей"

История запусков
uv run python -m src.main history list
uv run python -m src.main history show <run-id>

Evaluations
uv run python -m evals.component_evals
uv run python -m evals.system_evals
uv run jupyter lab evals/run_evals.ipynb
```

# Что нужно для финализации отчёта в PDF

---

## Шаг 1. Подготовить скриншоты по списку

---

Этот документ содержит 27 placeholder-маркеров вида `[СКРИН N: ...]`. Их нужно заменить на реальные изображения. Полный список и инструкции — в отдельном сообщении / файле `docs/REPORT_SCREENSHOTS.md` (см. чек-лист ниже).

Скриншоты по типу:

- **Скрины UI приложений** (Jaeger, Grafana, Langfuse, Prometheus, Jupyter) — после реального запуска системы.
- **Скрины терминала** — вывод CLI-команд.
- **Скрины файлов в IDE** — фрагменты кода и JSON в VS Code / Cursor / любой IDE.
- **Скрины-рендеры Mermaid-диаграмм** — отрисовать через `mermaid.live`, экспортировать PNG.

## Шаг 2. Положить скриншоты в папку

---

Создать `docs/screens/` и сложить туда файлы с именами `screen-01.png`, `screen-02.png`, ... `screen-27.png`. Имена должны соответствовать номерам placeholder'ов в этом документе.

## Шаг 3. Заменить placeholder'ы

---

В тексте отчёта строки вида `**[СКРИН 1: ...]**` заменить на полноценные markdown-картинки:

```
![Рис. 1. Архитектура мультиагентной системы](screens/screen-02.png)
```

Это можно сделать руками или одной командой `sed` (для каждого скрина).

## Шаг 4. Сгенерировать PDF

---

**Вариант А: Pandoc** (рекомендую — академический look)



```
Установка pandoc и LaTeX
Windows: choco install pandoc miktex
Linux: apt install pandoc texlive-xetex texlive-lang-cyrillic

Базовый рендер
pandoc docs/REPORT.md -o REPORT.pdf \
 --pdf-engine=xelatex \
 --listings \
 -V geometry:margin=2cm \
 -V mainfont="Times New Roman" \
 -V monofont="Consolas" \
 -V documentclass=report \
 --toc

С автокраскивающимися Mermaid-блоками (требуется mermaid-filter)
npm install -g mermaid-filter
pandoc docs/REPORT.md -o REPORT.pdf \
 --filter mermaid-filter \
 --pdf-engine=xelatex \
 --listings -V geometry:margin=2cm --toc
```

### Вариант Б: Турора (простой)

1. Открыть `docs/REPORT.md` в [Typora](#).
2. File → Export → PDF.
3. Турора сам рендерит Mermaid и подхватывает картинки относительно файла.

### Вариант В: VS Code Markdown PDF

1. Установить расширение «Markdown PDF» (yzane.markdown-pdf).
2. Открыть `REPORT.md`, F1 → «Markdown PDF: Export (pdf)».
3. Mermaid рендерится автоматически.

### Вариант Г: Online (если нет локально)

1. [stackedit.io](#) или [dillinger.io](#) — открыть содержимое `REPORT.md`.
2. Export → PDF.

## Список скриншотов: пронумерованный чек-лист

№	Что снять	Откуда	Статус
1a + 1b	Диаграмма классов (доменные типы + state/infra)	scripts/render_mermaid.py	✅ готово
2	High-level Mermaid из § «Архитектура»	scripts/render_mermaid.py	✅ готово
3	Фрагмент supervisor.py::build_graph()	IDE	✅ готово
4	Memory layers Mermaid	scripts/render_mermaid.py	✅ готово
5	Hybrid retrieval Mermaid	scripts/render_mermaid.py	✅ готово
6	Любой skills/*/SKILL.md с YAML frontmatter	IDE	✅ готово
7	Observability data flow Mermaid	scripts/render_mermaid.py	✅ готово
8	Вывод uv run python -m src.main check	Терминал	✅ готово
9	Вывод ... research "..." — начало (Run ID + таблица)	Терминал	✅ готово
10	Вывод ... research "..." — конец (отчёт + Verdict)	Терминал	✅ готово
11	runs/<id>/report.md в IDE с preview	VS Code / Obsidian	✅ готово
12	Jaeger UI — список trace'ов, выбрать один с раскрытыми spans	http://localhost:16686	✅ готово
13	Jaeger UI — детали span'a ollama.chat с атрибутами	Jaeger	✅ готово
14	Grafana — главный экран дашборда «Niche Research Agent»	http://localhost:3000	✅ готово
15	Grafana — зум на панель «Agent node duration p95»	Grafana	✅ готово
16	~~Langfuse — список trace'ов~~	~~http://localhost:3001~~	⊘ исключено (future work)
17	~~Langfuse — детали одного call~~	~~Langfuse~~	⊘ исключено (future work)
18	Prometheus — query llm_request_total с графиком	http://localhost:9090	✅ готово
19	Вывод ... component_evals — сводная таблица с порогами	Терминал	✅ готово
20	~~Вывод ... system_evals — таблица per-query~~	~~Терминал~~	⊘ исключено (system- evals run ~2- 3 ч на 1050 Ti — future work)

№	Что снять	Откуда	Статус
21	Jupyter — bar chart component metrics с порогами	jupyter lab	✅ готово
22	~~Jupyter — pie chart pass rate + verdicts + judge histogram~~	~~Jupyter~~	⊘ исключено (требуется system_evals — future work)
23	~~Jupyter — bar chart latency per query~~	~~Jupyter~~	⊘ исключено (требуется system_evals — future work)
24	Фрагмент data/mock_wb_products.json в IDE	IDE	✅ готово
25	knowledge_base/compliance/eac_certification.md в IDE	IDE	✅ готово
26	runs/<id>/state.json , секция prd	IDE	✅ готово
27	Вывод docker compose ps — 7 сервисов Up	Терминал	✅ готово

**Минимально нужно для убедительного отчёта:** 1, 2, 8, 9, 10, 11, 12, 14, 19, 21.

**Nice-to-have (показывают глубину):** 3, 4, 5, 6, 7, 13, 15, 18, 24, 25, 26, 27.

**Сознательно исключены:**

- **16, 17** (Langfuse) — см. §«Демонстрация работы», подсекция «Langfuse — LLM-tracing (текущий статус и границы реализации)». Функциональность покрывается OpenTelemetry-spans + JSONL-логами; полная интеграция Langfuse оставлена в future work.
- **20, 22, 23** (system-evals + Jupyter system-визуализации) — см. §«Демонстрация работы», подсекция «System-level». Полный прогон на 15-query наборе занимает ~2-3 часа на 1050 Ti; system-level качество демонстрируется через реальный end-to-end запуск (Рис. 8-12). Framework evals/system\_evals.py реализован и работоспособен — массовый прогон оставлен в future work.

## Готовые URL для скринов Mermaid-диаграмм

Открой [mermaid.live](https://mermaid.live), вставь нужный блок из § «Архитектура», § «Память и RAG», § «Observability» — экспорт «PNG» / «SVG», сохрани под именем screen-02.png (или соответствующим номером).

Чтобы получить чистый PNG без подложки редактора: в mermaid.live справа открой "Configuration" → "theme: neutral" + проверь «backgroundColor: white». Затем "Actions" → "PNG".

*Конец отчёта.*